



INDIVIDUAL PROJECT

CHENNAI MATHEMATICAL INSTITUTE

DEPARTMENT OF COMPUTING

Project Title

Cryptography & Information Theory

Author:
Koushan Saha

Supervisor:
Prof. Amit Kumar Sinhababu

May 27, 2024 - July 18, 2024

Acknowledgements

I am very grateful to my supervisor who contributed to the creation of this project, helpful discussions and useful comments. I am also grateful to Overleaf website(latex) which pointed out various inaccuracies and misprints, and suggested improvements in the presentation as well in the results. Needless to say, the responsibility for the remaining mistakes, as well as the responsibility for the (hopefully very few) new ones, is solely mine. It is a pleasure to thank Overleaf, for its great technical help and my supervisor for the guidance in arrangement of the pdf and giving great materials to study in the preparation of the final manuscript.

Contents

1	Introduction	3
1.1	Objectives	3
1.2	Challenges	3
1.3	Contributions	3
2	Background	4
3	Cryptography	5
3.1	Overview	5
3.2	Encryption	5
3.2.1	Shannon ciphers	6
3.2.2	Perfect Security	7
3.2.3	Two-time pad is insecure	8
3.2.4	No Integrity (Malleability)	8
3.3	Real-world Ciphers	8
3.4	Secure Cipher	9
3.4.1	PRG definitions	9
3.4.2	Semantic Security	10
3.5	Block Cipher	11
3.5.1	DES	12
3.5.2	Exhaustive Search	13
3.5.3	Quantum Attack	13
3.5.4	AES	13
3.5.5	Block cipher from PRG	14
3.5.6	Many-time key	14
3.5.7	Block cipher using random IV	15
3.5.8	Randomized Counter-mode	16
3.6	Message Integrity	16
3.6.1	Constructions	17
3.6.2	Collision Resistance	19
3.6.3	MAC from Hash functions	21
4	Information Theory	23
4.1	Data Compression	23
4.1.1	Symbol codes	24
4.1.2	Arithmetic Codes	25
4.2	Noisy Channel Coding	25
4.2.1	7,4 Hamming Code	27
4.3	Bayesian Inference	27
4.3.1	Clustering	28
4.3.2	Variational methods	29
4.4	Data Modelling with Neural Networks	30
4.4.1	Parallel Distributed Processing	30
4.4.2	Content addressable memory	31
4.4.3	Hopfield Network	31
5	Conclusion	33

Chapter 1

Introduction

Hello!!

1.1 Objectives

Information theory and Cryptography are very much important in Computer science and modern technologies. So our **goal** is to learn the basics of these topics and how to apply these into real life. Our objective is to use these theories in our real life.

1.2 Challenges

This project requires basic probability theory, coding theory and some amount of discrete mathematics. Since I had no knowledge of cryptography before, I have to start this project from the basic cryptography.

1.3 Contributions

I have used various internet sources to do this project. I have converted the project into pdf using Overleaf. Many digital copies of books are also used in this project. My Supervisor helped me to arrange the pdf properly and gave much study materials which are great and these materials helped me to complete the project.

Chapter 2

Background

Cryptography and Information theory are two closely related fields that have developed extensively over the past few centuries. Both deal with the protection and manipulation of information. The history of cryptography dates back to ancient civilizations. The **Egyptians** used simple substitution ciphers around 1900 BCE, and the **Spartans** used the **scytale transposition cipher**. **Julius Caesar's Caesar cipher**, a type of substitution cipher, one of the earliest known and simplest encryption techniques from this period. During middle age, the **Arab scholars** made contributions by doing analysis on **breaking substitution ciphers**. **Leon Battista Alberti and Blaise de Vigenère** introduced more sophisticated techniques. The **Enigma machine**, made by Germany in **WWII** was a significant cryptographic device of the 20th century and it's breaking by Allied cryptanalysts, including **Alan Turing**, had a major impact on the war's outcome. **Claude Shannon's** work laid the theoretical foundations of information theory and cryptography. **Whitfield Diffie and Martin Hellman** introduced the concept of **public-key cryptography**, leading to the development of the **RSA algorithm by Rivest, Shamir, and Adleman**. The **Data Encryption Standard (DES)** and later the **Advanced Encryption Standard (AES)** became widely adopted standards for data encryption. Cryptography now has various aspects of digital security, including **SSL/TLS for secure web browsing, end-to-end encryption in messaging apps, blockchain technology, and cryptocurrencies** like Bitcoin.

Information theory was founded by Claude Shannon in the 1940s. Information is measured in bits and represents the reduction in uncertainty when one learns the value of a random variable. Entropy is a measure of the uncertainty or unpredictability associated with a random variable. Information theory provides techniques to efficiently encode information so that it occupies less space (compression) while still retaining the essential information content. Information theory plays a crucial role in coding theory, which deals with the design of efficient and reliable methods of transmitting data across noisy channels. Error-correcting codes are essential in ensuring data integrity. Information theory finds applications in various fields including telecommunications, data compression, cryptography, statistical inference, and more recently in understanding complex systems like biological systems and even in quantum computing.

Chapter 3

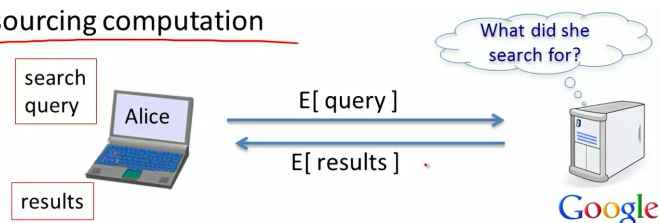
Cryptography

3.1 Overview

In this modern era, cryptography is used everywhere.

- **Secure Communication:** Web traffic is protected by the protocol HTTPS (Hypertext Transfer Protocol Secure) and Wireless traffic is protected by 802.11i/WPA2, GSM, Bluetooth. Let Alice is user and she is using some Web-server. Now to ensure no eavesdropping (stealing) and no tampering (modifying) of data, HTTP, more specifically SSL/TLS is used. SSL (Secure Sockets Layer)/TLS has 2 main parts. *Handshake protocol:* Alice & Bob share a secret key using public key cryptography, so that the attacker has no idea what the key is. *Record Layer:* Using that shared secret key they transmit data to maintain confidentiality and integrity.
- **Encrypting files on disk:** Encrypting files on disk is protected by EFS, TrueCrypt. Content in DVD and Blu-Ray is protected by CSS and AACS respectively. Here if we replace Bob by Alice then Alice can store a message and only Alice can read it later.
- **Symmetric Encryption:** It has two algorithms: Encryption and Decryption algorithms. Encryption algo takes the message m and secret key k and produces ciphertext c and sender sends it through the network. Decryption algo gets the ciphertext c and has secret key k , produces back the message m to the receiver. The encryption algo is publicly known.
- **Keys:** *One-time key:* If there is single message to send then it is encrypted by a new key, generated for every sending i.e., if we send message multiple times then every time it is encrypted by different keys (e.g. email). *Many-time key:* Same key is used to encrypt multiple messages.
- **Other Uses:** Encrypting *Digital Signatures*, anonymous communication, digital cash, preventing double spending of digital cash, elections, auctions etc.

Privately outsourcing computation

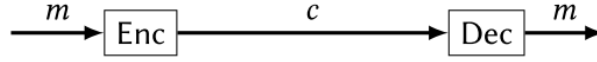


3.2 Encryption

Suppose Alice and Bob share a secret key k . Alice wants to send a message m over a network, maintaining secrecy of m in the presence of eavesdropping adversary. Think about another situation, where Alice wants to store some files on a disk so that no one else can read those files, only Alice can read those later. These problems arise the notion of secure communication. They need some techniques to maintain the secrecy.

3.2.1 Shannon ciphers

The mechanism for encrypting a message by shared secret key is called **Cipher**. A **Shannon Cipher** is a pair $P = (E, D)$ of functions where E is the encryption function which takes a key k and a message m as input and produces a **ciphertext** c as output, $c = E(k, m)$ (c is the encryption of m under k) and D is the decryption function which takes a key k and ciphertext c as input and produces the message m as output, $m = D(k, c)$ (m is the decryption of c under k).



Correctness Property: The functions E, D must satisfy the equation: $D(k, E(k, m)) = m$. Let KP = set of keys (key space), MP = set of messages (message space) and CP = set of ciphertexts (ciphertext space) then $E : KP \times MP \rightarrow CP$ and $D : KP \times CP \rightarrow MP$.

The idea is in the beginning, Alice and Bob somehow agree on a key $k \in KP$. Then Alice encrypts the message m she wants to send to Bob and sends the ciphertext $c = E(k, m) \in CP$ through the network. Bob gets c and decrypts it to the original message $m = D(k, c) \in MP$. We have to assume that c is not tampered with in transit from Alice to Bob.

Generally, keys, messages, ciphertexts are sequence of bytes. Keys are generally of fixed lengths. Messages and ciphertexts can be of fixed lengths or variable lengths.

One-time pad: It's a Shannon cipher $P = (E, D)$ and $KP = MP = CP = \{0, 1\}^L$ for some fixed parameter L , where $E(k, m) = k \oplus m$ and $D(k, c) = k \oplus c$. Here $\oplus$ is component-wise XOR.

Correctness Property: $D(k, E(k, m)) = D(k, k \oplus m) = k \oplus (k \oplus m) = (k \oplus k) \oplus m = 0^L \oplus m = m$.

Variable length One-time pad: $KP = \{0, 1\}^L$ and $MP = CP = \{0, 1\}^{\leq L}$ for some fixed parameter L , where $E(k, m) = k[0, \dots, l-1] \oplus m$ and $D(k, c) = k[0, \dots, l-1] \oplus c$, if m, c have length l . We can prove the correctness property similarly as above fixing the length l of the message m .

Substitution Cipher: Let Σ is a finite set symbols (e.g. A-Z with " "). $MP = CP = \Sigma^L$ for fixed length L . KP = all permutations of the set Σ , so $|KP| = |\Sigma|!$ (e.g. $|KP| = 27! \approx 1.09 \cdot 10^{28}$). So each element k of KP is a bijective function which maps each symbol to another symbol. Choose a particular $k \in KP$ then $E(k, m) = (k(m[0]), k(m[1]), \dots, k(m[L-1]))$ and $D(k, c) = (k^{-1}(c[0]), k^{-1}(c[1]), \dots, k^{-1}(c[L-1]))$

Correctness Property:

$D(k, E(k, m)) = (k(E(k, m)[0]), \dots, k(E(k, m)[L-1])) = (k(k^{-1}(m[0])), \dots, k(k^{-1}(m[L-1]))) = m$

e.g. Caesar Cipher is substitution cipher where substitution is shifting by 3, i.e., $a \rightarrow d, b \rightarrow e, \dots, y \rightarrow b, z \rightarrow c$.

Methods to break Substitution Cipher: We know the most frequent letter in English text is "e" (12.7%). Count the occurrences of the letters in the given ciphertext. Then the most common letter in ciphertext is the encryption of letter "e" with very high probability. Next most common letters in English are "t" (9.1%), "a" (8.1%). Similarly before get the encryption of "t" and "a". Then use the frequency of pair of letters. We know the most common pairs of letters are: "he", "an", "in", "th" and so on. By trial and error we can figure out the encryption key for substitution cipher. So any ciphertext attacker can break the encryption of substitution cipher.

Additive One-time pad: $KP = MP = CP = \{0, \dots, n-1\}$ for some $n \in \mathbb{N}$.

$E(k, m) = k + m \bmod n$ and $D(k, c) = c - k \bmod n$.

Correction Property: $D(k, E(k, m)) = E(k, m) - k = k + m - k = m \bmod n$.

e.g. *Vigener Cipher:* $k = \text{CRYPTO}$. Replicate k upto the length of the message then add them modulo $n = 26$. Vigener cipher is easy to break. We have the ciphertext and it's length. We know the key CRYPTO. So we replicate the word upto the length of ciphertext and subtract it from ciphertext modulo 26 to get the message.

Data Encryption Standard (DES): It has key space $= 2^{56}$ and block size 64 bits, which is not enough on present days, it's encryption can be broken by brute force computation. These days AES (128 bit keys), Salsa20 are used for encryption.

3.2.2 Perfect Security

We want that if the eavesdropping adversary obtains the ciphertext, it would be hard to guess the message. So for hard guessing we need to choose the key k randomly from a very large key space so that the probability of guessing is very low. If there exists keys k_0, k_1 such that $E(k_0, m_0) = E(k_1, m_1) = c$ then either $m = m_0$ or $m = m_1$. For example, if for 100 keys k_0 and 200 keys k_1 , $E(k_0, m_0) = E(k_1, m_1) = c$ then the probability that the guess is correct $= \frac{200}{100+200} = 66.67\% \geq 50\%$ without knowing the ciphertext at all. Perfect security rules out the possibility that knowledge of the ciphertext increases the probability of guessing the encrypted message.

Definition: $P = (E, D)$ is Shannon cipher defined over (KP, MP, CP) . Let random variable k is uniformly distributed over KP . If $\forall m_0, m_1 \in MP$ and $\forall c \in CP$: $Pr[E(k, m_0) = c] = Pr[E(k, m_1) = c]$ then we say P is perfectly secure Shannon cipher.

Theorem1: TFAE: (i) P is perfectly secure, (ii) $\forall c \in CP \exists N_c$ s.t. $\forall m \in MP$: $|\{k \in KP : E(k, m) = c\}| = N_c$, (iii) If random variable k is uniform distribution over KP , then each random variables $E(k, m)$ for $m \in MP$ has the same distribution.

Pf: (ii) is equivalent to (iii) as $Pr[E(k, m) = c] = P_c = \frac{N_c}{|KP|}$.

(i) $\Rightarrow$ (ii): Let $c \in CP, m_0 \in MP, c \in CP$: $Pr[E(k, m) = c] = P_c \Rightarrow |\{k \in KP : E(k, m) = c\}| = P_c \cdot |KP| = N_c$. Similarly (ii) $\Rightarrow$ (i).

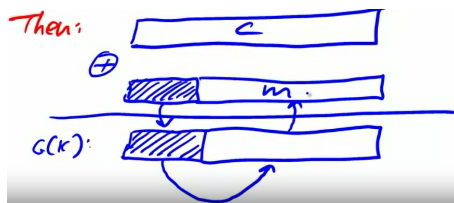
Theorem2: One-time pad is a perfectly secure shannon ciphertext.

Pf: For each $c \in CP$ and for each $m \in MP$: the key $m \oplus c = m \oplus (m \oplus k) = 0 \oplus k = k$, so $N_c = 1$.

Variable length One-time pad is not perfectly secure since if m_0, c have length 1 and m_1 has length 2 then $Pr[E(k, m_0) = c] = 0.5$ but $Pr[E(k, m_1) = c] = 0$. Similar for substitution cipher.

Theorem3 (Shannon's Thm): Perfect secrecy $\Rightarrow |KP| \geq |MP|$.

Making OTP for practical use (Stream Cipher): We replace random key by "pseudorandom" key. PRG (pseudorandom generator) is a function $G : \{0, 1\}^s \rightarrow \{0, 1\}^n$ where $\{0, 1\}^s$ is called the seed space and $n \gg s$. The generator G is efficiently computable by a deterministic algorithm. Now choose a key $k \in$ seed space then define $c = E(k, m) = m \oplus G(k)$ and $D(k, c) = c \oplus G(k)$. Let PRG is predictable, i.e., $\exists i$ s.t. if we know $G(k)[0..i]$ then by some algo we get entire $G(k)$.



If we know $m[0..i]$ then we get $G(k)[0..i] = m[0..i] \oplus c[0..i]$ then we predict the entire $G(k)$ then get $m = c \oplus G(k)$. So PRG must be unpredictable.

Definition: We say $G : K \rightarrow \{0, 1\}^n$ is predictable if $\exists$ an efficient algo A and $1 \leq i < n$ s.t. $\Pr[k \in KP : A(G(k)[0..i]) = G(k)[i+1]] \geq \frac{1}{2} + \epsilon$ where $\epsilon \geq \frac{1}{2^{30}}$.

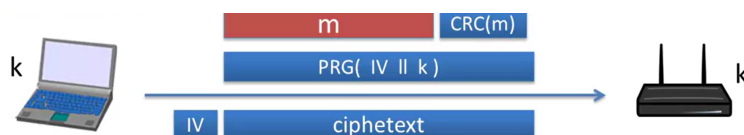
Linear Congruential Generator (Weak PRG): It has parameters a, b, p where p is prime. $r[0] = seed, r[i] = a \cdot r[i-1] + b \pmod{p}$. It is easy to predict.

Glibc Generator (Weak PRG): Here $r[i] = r[i-3] + r[i-31] \pmod{2^{32}}$ and output $r[i] >> 1$. It is also easy to predict.

3.2.3 Two-time pad is insecure

We can't use the stream cipher key more than once. Let we have generated $c_1 \leftarrow m_1 \oplus PRG(k)$ and $c_2 \leftarrow m_2 \oplus PRG(k)$. Now $c_1 \oplus c_2 = m_1 \oplus PRG(k) \oplus m_2 \oplus PRG(k) = m_1 \oplus m_2 \oplus (PRG(k) \oplus PRG(k)) = m_1 \oplus m_2$. Because of much redundancy in English and ASCII encoding, we can get m_1, m_2 from $m_1 \oplus m_2$ which is known to us. So stream cipher is always one-time pad and the same key can't be used more than once.

Examples: (i) **Project Venona** by US used above method to break the encryption of two-time pad. So generate a new key for every session (e.g. TLS). (ii) **MS-PPTP** is windows client-server communication system, where the messages from client to server and server to client can't be encrypted by the same stream cipher key. So the system should have a pair of keys $(k_{c \rightarrow s}, k_{s \rightarrow c})$ and this pair would be available to both client and server. Then the system will be secure. (iii) **802.11b WEP** is a wifi security protocol. Where the encryption is the following:



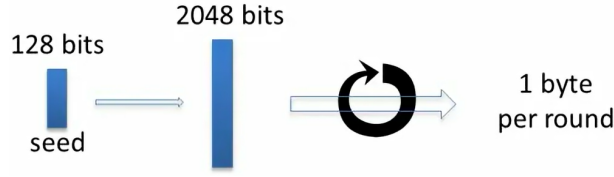
Here length of IV is 24 bits and k is of length 104 bits, so total 128 bits. The computer sends the ciphertext IV concatenated with the ciphertext obtained by XOR of $m || CRC(m)$ and $PRG(IV || k)$, IV is variable string. So # of strings for IV = $2^{24} \approx 16M$ frames. After 16M frames the same key is used for encryption, which makes it insecure. To resolve it, we break $PRG(k)$ into several segments and each segment is used to encrypt one specific frame. (iv) Disk encryption has got the same problem. Let we saved files for Bob as: "To Bob:...". Next time, for Alice we saved as: "To Alice:...". If the attackers get access of both ciphertexts, then they would see that only some parts of ciphertexts are changed. From that they would get the location of the header of the disk. That's make the files insecure. So don't use stream cipher for disk encryption.

3.2.4 No Integrity (Malleability)

Take the example OTP. Let Bob sends a message $m = \text{"From: Bob"}$, then ciphertext would be $m \oplus k$. The attacker does some XOR operation with ciphertext corresponding to the name "Bob" so that when it is decrypted it becomes the message "From: Alice" instead of "From: Bob" and this modification of ciphertext is undetected. This can create huge problems.

3.3 Real-world Ciphers

1. **RC4:** It's an old software, made in 1987. It has seed of 128 bits, which maps to strings of 2048 bits and generates 1 byte per round.

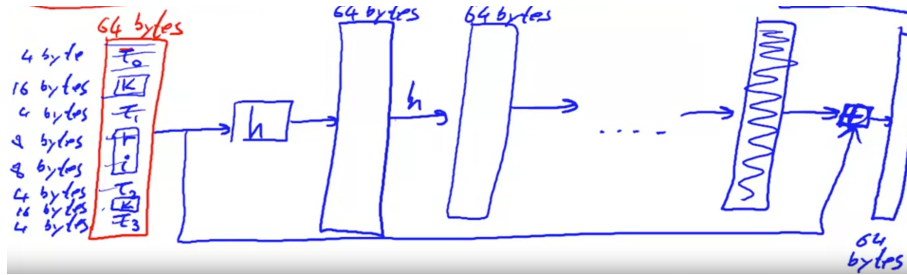


It has some weaknesses like: biased in initial inputs and it can have relatable key attacks.

2. **CSS:** This encryption is based on hardware. It uses **Linear Feedback Shift Register (LFSR)**, which is divided into cells and each cell is of 1 bit. Then some cells of some specific positions (taps) have XOR operation and after each clock-cycle the cells shift to right and the rightmost cell falls off and the leftmost bit becomes the XOR of those bits. So here the seed is the initial state of the LFSR. e.g. DVD encryption (CSS) uses 2 LFSRs, GSM encryption (A5/1,2) uses 3 LFSRs, Bluetooth uses 4 LFSRs. But all of these are already broken.

Breaking CSS: CSS has seed of 5 bytes = 40 bits. It uses 17 bit LFSR and 25 bit LFSR and the keys are 1||1st 2 bytes of 1st LFSR and 1||last 3 bytes of 2nd LFSR. Each LFSR generate 8 bits in each clock-cycle and these strings are added modulo 256 and add the carries from the previous blocks. So CSS generates 1 byte per round. Like DVD encryption, attacker knows some prefix of the plain text, say n bits, then we XOR that prefix with ciphertext[0.. n] and get the 1st n bits of PRG, i.e., 1st n bits of the output of CSS. Then we try all 2^{17} possibilities of 1st LFSR and for each possible values we run the LFSR for the 1st n bits. Now we get 1st n bits of the output of the 2nd LFSR by subtracting plain text n bits to 1st LFSR output's n bits. Then we verify that n bit string came from 2nd LFSR or not. If not then try another possible value for 1st LFSR. If it is then we get the initial states of both LFSRs. Then we predict the CSS output and the the plain text by XOR operation between CSS output and ciphertext. So it takes at most 2^{17} times to break encryption.

3. **Modern Stream Cipher (eStream):** $PRG : \{0, 1\}^s \times R \rightarrow \{0, 1\}^n$ where $\{0, 1\}^s = \text{seed}$, $n \gg s$ and R is a set of non-repeating value for a given key. Now $E(k, m, r) = m \oplus PRG(k, r)$, here the pair (k, r) never repeats so that we can reuse the same key. e.g. **Salsa20** which can be implemented as both hardware and software. $Salsa20 : \{0, 1\}^{128 \text{ or } 256} \times \{0, 1\}^{64} \rightarrow \{0, 1\}^n$ (max n is 2^{73} bits). Now, $Salsa20(k, r) = H(k, (r, 0)) || H(k, (r, 1)) || H(k, (r, 2)) || \dots$ a long string. Now the function H is defined as follows:



Take h an invertible function and apply h 10 times on the input string $\tau_0 || k || \tau_1 || r || i || \tau_2 || k || \tau_3$ of 64 bytes where τ_i -s are constants of 4 bytes, k is the key of 16 bytes, r is the non-repeating value of 8 bytes and i is the index of the counter of 8 bytes. The string, we get, of 64 bytes applying h 10 times, is added to the input string. That is the output string of function H .

3.4 Secure Cipher

3.4.1 PRG definitions

Definition1: Statistical test on $\{0, 1\}^n$ is an algo A s.t. $A(x)$ outputs 0 or 1. Here $A(x) = 0$ means x is not random and $A(x) = 1$ means x is random. e.g. (i) $A(x) = 1 \iff |0(x) - 1(x)| \leq 10 \cdot \sqrt{n}$, (ii) $A(x) = 1 \iff |11(x) - \frac{n}{4}| \leq 10 \cdot \sqrt{n}$, (iii) $A(x) = 1 \iff \max \text{run of } 1(x) \leq 10 \cdot \log_2 n$.

Definition2: Let $G : K \rightarrow \{0, 1\}^n$ be a PRG and A is a statistical test on $\{0, 1\}^n$. The advantage of A on G is defined as the quantity $Adv[A; G] = |Pr_{k \leftarrow K}[A(G(k)) = 1] - Pr_{r \leftarrow K}[A(r) = 1]| \in [0, 1]$.

So if $Adv[A; G]$ is close to 1 then the algo A behaves very differently outside $G(k), k \in K$, so algo A can distinguish G from random. If $Adv[A; G]$ is close to 0 then algo A can't distinguish G from random. e.g. If G is such that $msb(G(k)) = 1$ for $\frac{2}{3}$ rd of the keys and $A(x) = 1 \iff msb(x) = 1$ then $Adv[A; G] = \frac{2}{3} - \frac{1}{2} = \frac{1}{6}$.

Definition3: $G : K \rightarrow \{0,1\}^n$ is secure PRG if no efficient algo can distinguish it from random i.e., for every algo A $Adv[A; G]$ is close to 0, negligible. If we can prove some PRG is secure $\implies P \neq NP$. So we don't know any secure PRG till now.

Fact: A secure PRG is unpredictable.

Pf: Let PRG G is predictable. So there exists algo A s.t. $A(G(k)[0..i]) = G(k)[i+1] \implies Pr[A(G(k)[0..i]) = G(k)[i+1]] = \frac{1}{2} + \epsilon$ where ϵ is non-negligible. Define: statistical test B s.t. $B(x) = 1$ if $A(x[0..i]) = x_{i+1}$ otherwise 0. Then $Adv[B; G] = \frac{1}{2} + \epsilon - \frac{1}{2} = \epsilon$, non-negligible. Then A can distinguish G from random, so insecure.

Theorem1: Unpredictable PRG is secure.

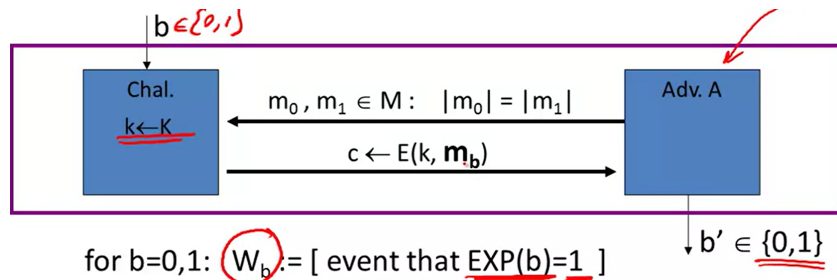
Definition4: Suppose P_1, P_2 are distributions over $\{0,1\}^n$. We say that P_1 is computationally indistinguishable from P_2 (denoted as $P_1 \approx_P P_2$) if $\forall$ efficient statistical tests A $|Pr_{x \leftarrow P_1}[A(x) = 1] - Pr_{x \leftarrow P_2}[A(x) = 1]| < \text{negligible}$.

3.4.2 Semantic Security

Shannon's perfect secrecy:

(E, D) is perfectly secure if $\forall m_0, m_1; k \in K: E(k, m_0) \approx_P E(k, m_1)$.

Semantic Security (One-time key):



Suppose we have 2 experiments $E(0), E(1)$ and $b = 0, 1$ and $W_b = \text{event with } E(b) = 1$. So we have two messages $m_0, m_1 \in M$. There is a challenger who picks an arbitrary key $k \in K$. If the experiment is $E(0)$ then challenger outputs the encryption of m_0 , otherwise encryption of m_1 . There is an adversary who has access to the ciphertexts and tries to predict whether the given ciphertext is encryption of m_0 or m_1 . Now $Adv_{SS}[A; E] = |Pr[W_0] - Pr[W_1]| \in [0, 1]$. If $Adv_{SS}[A; E]$ is close to 0 then the adversary can't distinguish between W_0, W_1 , so can't predict the encryption. If $Adv_{SS}[A; E]$ is close to 1 then the adversary can distinguish between W_0, W_1 , so can predict the encryption.

Definition1: The semantic security E is secure if $\forall$ efficient adversary A : $Adv_{SS}[A; E]$ is negligible i.e., $\forall m_0, m_1 \in M: \{E(k, m_0)\} \approx_P \{E(k, m_1)\}$.

Example: Let there exists an adversary A which can detect efficiently the $LSB(m)$ from ciphertext. Suppose $LSB(m_0) = 0, LSB(m_1) = 1$ for $m_0, m_1 \in M$. Let we are adversary B . We get the ciphertext generated by the challenger and run the adversary A on given ciphertext. Then

$Adv_{SS}[B; E] = |Pr[E(0) = 1] - Pr[E(1) = 1]| = |0 - 1| = 1$ since for $E(0)$ message m_0 is passed and $LSB(m_0) = 0$ so $Pr = 0$ and for $E(1)$ message m_1 is passed and $LSB(m_1) = 1$ so $Pr = 1$. So $E' = (E, D)$ is not semantically secure.

Fact: If there exists an adversary A which can detect some property (information) of the strings from ciphertext for some encryption E , then (E, D) is not semantically secure.

Theorem1: OTP is semantically secure.

Pf: Let $m_0, m_1 \in M$ and challenger gets key $k \in K$ then the ciphertexts would be $k \oplus m_0, k \oplus m_1$. Now we know for OTP the XOR operation has uniform distribution over CP . So $\forall$ adversary A : $Pr[A(E(k, m_0)) = 1] = Pr[A(E(k, m_1))] \implies Adv_{SS}[A; E] = 0$.

Theorem2: $G : K \rightarrow \{0, 1\}^n$ is secure PRG $\implies$ stream cipher E generated by G is semantically secure.

Pf: We will prove: $\forall$ sem sec adv $A \exists$ PRG adversary B s.t. $Adv_{SS}[A; E] \leq 2 \cdot Adv_{PRG}[B; G]$. Now for PRG encryption is: $m_b \oplus G(k)$ where key k is given by the challenger. Now we change the encryption to $m_b \oplus r$, where r is totally random key. As G is secure, no efficient algo can distinguish this change i.e., Adv_{SS} is negligible. Now the encryption becomes OTP. For OTP, $Adv_{SS} = 0$. So the total Adv_{SS} is negligible. So the encryption is sem. sec. If we do it rigorously, then define $W_b = \text{event for } b' = 1$, where $b = 0, 1$ and b' is the output of the adversary A . For random pad $R_b = \text{event for } b' = 1$. $|Pr[R_0] - Pr[R_1]| = Adv_{SS}[A; OTP] = 0 \implies Pr[R_0] = Pr[R_1]$. Now if $|Pr[W_b] - Pr[R_b]| = Adv_{PRG}[B; G]$ for $b = 0, 1$ and for some B . Then by triangle inequality, $|Pr[W_1] - Pr[W_0]| \leq |Pr[W_0] - Pr[R_0]| + |Pr[W_1] - Pr[R_1]| = 2 \cdot Adv_{PRG}[B; G] \implies Adv_{SS}[A; G] \leq 2 \cdot Adv_{PRG}[B; G]$. Then since G is secure so $Adv_{PRG}[B; G]$ is negligible $\implies Adv_{SS}[A; G]$ is also negligible. So stream cipher is sem. sec. Need to show: $\exists B$ s.t. $|Pr[W_b] - Pr[R_b]| = Adv_{PRG}[B; G]$ for $b = 0, 1$. Take $y \in \{0, 1\}^n$ and we become the adversary. Given $m_0, m_1 \in M$, A gets ciphertexts and returns b' . $Adv_{PRG}[B; G] = |Pr_{r \leftarrow \{0, 1\}^n}[B(r) = 1] - Pr_{k \leftarrow K}[B(G(k)) = 1]| = |Pr[R_b] - Pr[W_b]|$. So we get the adversary B . Done!

ECB (Electronic Code Book): We have blocks of messages and we do encryption one block by another and merge them to get the ciphertext. So if $m_1 = m_2$ then $c_1 = c_2$. We prove that ECB is not semantically secure. Take two plain texts: "Hello World", "Hello Hello". The ciphertexts are: $c_1 = a_1 || b_1$ and $c_2 = a_2 || b_2$. Then if $a_1 = b_1$ then adversary knows the plain text is: "Hello Hello", otherwise "Hello World". So $Adv[A; ECB] = |1 - 0| = 1$. Not secure!

3.5 Block Cipher

Block cipher has two functions E, D and E takes an plaintext m of length n and using a key k of certain length produces ciphertext c of length n . The encryption is done by iteration. At first key k is divided into p parts $k_1, \dots, k_p$. There is a round function $R(k, m)$. Then the plaintext m is passed to this function as, $m \rightarrow R(k_1, m) = m_1; m_1 \rightarrow R(k_2, m_1); \dots; m_{p-1} \rightarrow R(k, m_{p-1}) = m_p$. m_p is the ciphertext. Examples: 3DES: $n = 64$ bits, $k = 168$ bits ($p = 48$), AES: $n = 128$ bits, $k = 128, 192, 256$ bits, AES128 ($p = 10$).

Definition1: Pseudo random function (PRF) defined over (K, X, Y) : $F : K \times X \rightarrow Y$ such that exists efficient algo to evaluate $F(K, X)$.

Definition2: Pseudo random permutation (PRP) defined over (K, X) : $E : K \times X \rightarrow X$ such that: (i) exists efficient deterministic algo to evaluate $E(K, x)$, (ii) the function $E(k, \cdot)$ is one-to-one, (iii) exists efficient inversion algo $D(k, y)$. **Examples:** 3DES ($K = \{0, 1\}^{168}$, $X = \{0, 1\}^{64}$), AES ($K = X = \{0, 1\}^{128}$). **Fact:** Any PRP is a PRF where $X = Y$.

Definition3: A PRF is secure if a random function in $Func[X, Y]$ is indistinguishable from a random function in $S_F = \{F(k, \cdot) | k \in K\}$.

Theorem1: If $F : K \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ is a secure PRF then $G : K \rightarrow \{0, 1\}^{nt}$ is secure PRG where $G(k) = F(k, 0) || F(k, 1) || \dots || F(k, t-1)$.

Pf: F is PRF so $F(k, \cdot)$ is indist from $f(\cdot)$. So $f(0) || f(1) || \dots || f(t-1)$ is indist from $F(k, 1) || \dots || F(k, t-1) = G(k)$.

PRF Switching Lemma: If $|X|$ is large enough then any secure PRP is secure PRF.

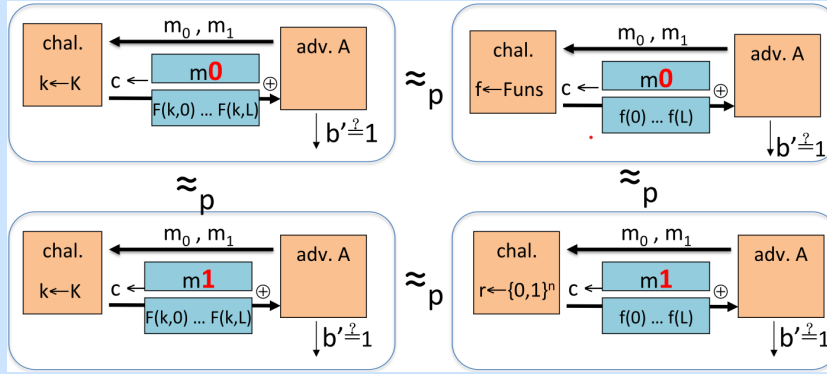
Lemma: Let E be PRP over (K, X) then for any q query adversary A :

$|Adv_{PRF}[A; E] - Adv_{PRP}[A; E]| < \frac{q^2}{2|X|}$ (negligible if $|X|$ is very large). Also $Adv_{PRF}[A; E]$ negligible $\Rightarrow Adv_{PRP}[A; E]$ negligible.

Secure construction of ECB: Suppose we have a PRF F and plain text $m[0] || m[1] || \dots$ and key k . Define $E(k, m) = c[0] || c[1] || \dots$, where $c[i] = m[i] \oplus F(k, i)$.

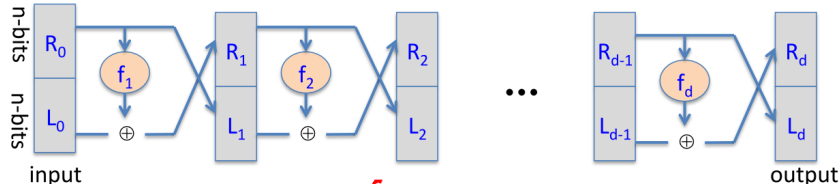
Thm for Det. Counter Mode security: For any $L > 0$ if F is secure PRF over (K, X, X) then E is sem. sec. cipher over (K, X^L, X^L) , i.e., for any eff adv A attacking $E \exists$ eff adv B and n s.t. $Adv_{SS}[A; E] = 2 \cdot Adv_{PRF}[B; F]$.

Pf: F secure PRF $\Rightarrow Adv_{PRF}[B; F]$ is negligible $\Rightarrow Adv_{SS}[A; E]$ is negligible.



3.5.1 DES

Constructed using Feistel network. Take arbitrary functions $f_1, \dots, f_d : \{0, 1\}^n \rightarrow \{0, 1\}^n$



Now the input is $R_0 || L_0$, $2n$ bits. $R_i = f_i(R_{i-1}) \oplus L_{i-1}$ and $L_i = R_{i-1}$ for $i = 1, \dots, d$. We will show $\exists$ an invertible function $F : \{0, 1\}^{2n} \rightarrow \{0, 1\}^{2n}$. For one transition $R_i \rightarrow R_{i+1}$ and $L_i \rightarrow L_{i+1}$, $R_i = L_{i+1}$ and $L_i = f_i(L_{i+1}) \oplus R_{i+1}$. So we get the inverse f_i^{-1} for transition f_i . Repeatedly doing inversion from f_d to f_1 , we get the inverse. So we get the decryption circuit.

Theorem2: $f : K \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ secure PRF $\Rightarrow F : K^3 \times \{0, 1\}^{2n} \rightarrow \{0, 1\}^{2n}$ secure PRP.

Here in each 3 transition different keys are used. $F(k_0, R_0), F(k_1, R_1), F(k_2, R_2)$ where k_i 's are distinct keys. DES is 16 round Feistel network where $f_i : \{0, 1\}^{32} \rightarrow \{0, 1\}^{32}$, $f_i(x) = F(k_i, x)$, k_i 's are round keys of 48 bits derived from K for $i = 1, \dots, 16$. There is a construction for $F(k_i, x)$.

3.5.2 Exhaustive Search

Our goal is: given the pair of input output string (m, c) find the key k .

Lemma: Suppose DES is an ideal cipher (2^{56} random invertible functions from $\{0, 1\}^{64}$ to $\{0, 1\}^{64}$) then $\forall m, c$ there is at most one key k s.t. $c = DES(k, m)$ with probability $\geq 1 - \frac{1}{2^{56}} \approx 99.5\%$.

Pf: $Pr[\exists k \neq k' | c = DES(k, m) = DES(k', m)] \leq \sum_{k' \in \{0, 1\}^{56}} Pr[DES(k, m) = DES(k', m)] = 2^{56} \cdot \frac{1}{2^{64}} = \frac{1}{2^{56}}.$

Examples: For 2 DES pairs $(m_1, c_1 = DES(k, m_1)), (m_2, c_2 = DES(k, m_2))$ probability $\approx 1 - \frac{1}{2^{71}}$. For AES-128 2 pairs: probability $\approx 1 - \frac{1}{2^{128}}$. These pairs are enough to do exhaustive key search.

Breaking DES: In DES, the message is broken into several parts each of 64 bits and with a particular key k each part is converted into ciphertext to get the actual ciphertext. The goal is to find the key k for this encryption. In 1997, internet search broke it in 3 months. In 1998 EFF machine with deep crack broke it in 3 days. In 1999, combined search broke it in just 22 hours. So we can't use 56 bit cipher.

Strengthening the encryption: (i) 3DES: $3E : K^3 \times M \rightarrow M$ s.t. $3E((k_1, k_2, k_3), m) = E(k_1, D(k_2, E(k_3, m)))$. If $k_1 = k_2 = k_3$ then 3DES=DES. Key size of 3DES is $3 \times 56 = 168$ bits. Simple attack on this takes time $\approx 2^{118}$. (ii) Double-DES: $2E((k_1, k_2), m) = E(k_1, E(k_2, m)) = c$. Key-length 112 bits. Let $M = (m_1, \dots, m_{10})$ and $C = (c_1, \dots, c_{10})$. Goal: Find k_1, k_2 s.t. $E(k_1, E(k_2, M)) = C \Rightarrow E(k_2, M) = D(k_1, C)$. Build a table for keys of 56 bits with 2nd column $E(k', M)$ and sort on 2nd column and search for each possible keys. Total computation takes time $2^{56} \cdot \log(2^{56})$. For each $k \in \{0, 1\}^{56}$ we test $D(k, C)$ in 2nd column. If so then $E(k_i, M) = D(k, C)$ then $(k_1, k_2) = (k, k_i)$. Total time taken $2^{56} \cdot \log(2^{56}) + 2^{56} \cdot \log(2^{56}) < 2^{63} \ll 2^{112}$. Same attack on 3DES takes time $2^{118} > 2^{112}$, (iii) Define $EX((k_1, k_2, k_3), m) = k_1 \oplus E(k_2, m \oplus k_3)$ where E is block cipher. Key-length = $64 + 56 + 64 = 184$ bits. Simple attack takes time $2^{56+64} = 2^{120}$.

Other Attacks: (i) **Side Channel attack:** If we have a smartcard for payments measure the time, power of enc/dec of the card. We get a graph for the time, power consumption. There we can distinguish the IP parts and each round of the 16 rounds. From that we can actually detect secret the key of enc. (ii) **Fault attack:** Computing errors in the last round expose the secret key for enc. (iii) **Linear Cryptanalysis:** Suppose we have $k, m, c = E(k, m)$ and $Pr[m[i_1, \dots, i_r] \oplus c[j_1, \dots, j_v] = k[l_1, \dots, l_u]] = \frac{1}{2} + \epsilon$, then somehow the key is dependent on message and ciphertext. For DES $\epsilon = \frac{1}{2^{21}} \approx 0.0000000477$.

Theorem1: Given $\frac{1}{\epsilon^2}$ random pairs of (m, c) $k[l_1, \dots, l_u] = MAJ[m[i_1, \dots, i_r] \oplus c[j_1, \dots, j_v]]$ with probability $\geq 97.7\%$. Example: For DES $\epsilon = \frac{1}{2^{21}}$ then 2^{42} pairs can find $k[l_1, \dots, l_u]$ in 2^{42} time. Experimentally, $u = 14$. Then remaining $56 - 14 = 42$ bits of key has 2^{42} possible values. By bruteforce search, we can get remaining in 2^{42} time. So total time = $2^{43} \ll 2^{56}$.

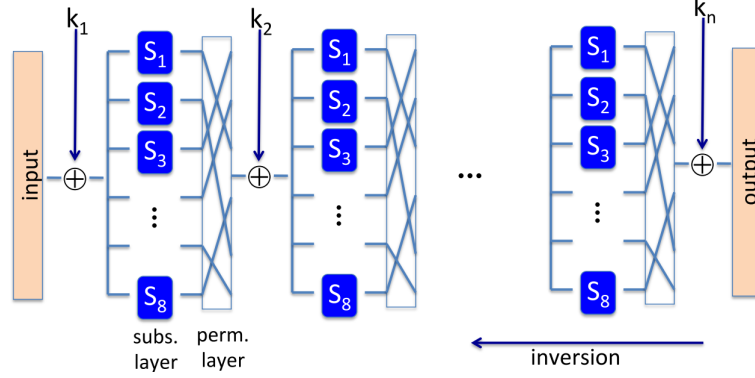
3.5.3 Quantum Attack

Generic search problem: Let $f : X \rightarrow \{0, 1\}$ then find $x \in X$ s.t. $f(x) = 1$. Classical computer takes $O(|X|)$. Quantum computer takes $O(\sqrt{|X|})$ (Grover 1996).

Quantum Exhaustive Search: Given $m, c = E(k, m)$ define for $k \in K$ $f(k) = 1$ if $c = E(k, m)$ otherwise 0. Grover told that we can find the key in $O(\sqrt{|K|})$. So for DES $\approx 2^{28}$, for AES-128 $\approx 2^{64}$. So for quantum computer we need 256 bits secret key which takes $\approx 2^{128}$, which is secure.

3.5.4 AES

Block size: 128 bits and key sizes: 128, 192, 256 bits. AES has substitution permutation network.



This structure has several rounds and round keys. Each round has some blocks in substitution layer and this blocks are permuted in permutation layer and this whole process is invertible.

AES-128: It has keys of 128 bits = 16 bytes. We think of keys as a 4×4 matrix where each cell is of 1 byte. Now there are 10 rounds. First 9 rounds have the functions: (i) ByteSub, (ii) ShiftRow, (iii) MixColumn. Last round has only (i) and (ii). Input and output both are 4×4 array. Key length = 16 bytes $\times$ 11 = 176 bytes since 11 keys are required for 10 rounds. ByteSub is 1 byte S-box with 256 bytes table i.e., for each i, j : $A[i][j] \leftarrow s(A[i][j])$. ShiftRow is a function which shifts a row by 1/2/3 cell to the right. MixColumn is a multiplying a column by scalar or adding it to another column. Example: JavaScript AES (in browser).

AES Hardwares: (i) aesenc, aesenclast has 1 round, 128 bits registers: xmm1 (state), xmm2 (round key). $\text{aesenc xmm1 xmm2} \Rightarrow \text{result in xmm1}$. (ii) aeskeygenassist performs AES key expansion.

Attacks: The best attack on AES is 4 times better than exhaustive search. Related key attack takes 2^{99} input pairs from 4 related keys and recovers the keys in time $\approx 2^{99}$.

3.5.5 Block cipher from PRG

Let $G : K \rightarrow K^2$ is secure PRG. Define $G_1 : K \rightarrow K^4$ as $G_1(k) = G(G(k)[0]) || G(G(k)[1])$. Define 1-bit PRF $F : K \times \{0, 1\} \rightarrow K$ as $F(k, x) = G(k)[x]$. Define 2-bit PRF $F_1 : K \times \{0, 1\}^2 \rightarrow K$ as $F_1(k, x) = G_1(k)[x]$. Similarly define $G_2 : K \rightarrow K^8$ and we get 3-bit PRF $F_2 : K \times \{0, 1\}^3 \rightarrow K$.

Theorem1: If G is secure PRG then F is secure PRF (Pf is from definition).

Theorem2: If G_1 is secure PRG then F_1 is secure PRF.

Pf: We get $G(k)[0], G(k)[1]$ at first. We can replace them by truly random keys r_0, r_1 since F is secure PRF. Then we can replace $G_1(k)[00], G_1(k)[01], G_1(k)[10], G_1(k)[11]$ by $r_{00}, r_{01}, r_{10}, r_{11}$ in the next level since G_1 is secure. So we get $F_1(k) \approx_P$ random keys in K^4 .

GGM PRF: Let $G : K \rightarrow K^2$ is secure PRG. Define $F : K \times \{0, 1\}^n \rightarrow K$ as given $x = x_0 x_1 \dots x_{n-1} \in \{0, 1\}^n$, $F(k, x) = G(G(\dots(G(k)[x_0])[x_1]) \dots [x_{n-1}])$. G secure PRG $\Rightarrow F$ secure PRF on $\{0, 1\}^n$. It is much slow in performance. We can build secure PRP from PRF by just plugging GGM PRF into Luby-Rackoff thm (Feistel network).

3.5.6 Many-time key

Here the same key is used in multiple messages. Adversary has the power of **chosen plaintext attack (CPA)**. For example: adversary can send an email to Alice and Alice encrypts the email using the same key and stores it on a disk. But the adversary can access the ciphertext of the encryption. From that adv can derive the secret key for Alice encryption. For CPA, semantic security is the following: m_0, m_1 are messages and challenger chooses random key k and for experiment 0 it sends c_0 the encryption of m_0 , for experiment 1 it sends c_1 the encryption of m_1 . The adversary can send chosen pair of messages $(m_{i,0}, m_{i,1})$ and it gets the encryption of the of them. Adversary has to decide which experiment is this 0 or 1. Adversary can do q queries with total q pairs of messages. If adv wants the encryption of a particular message m then it can

send the pair $(m_{i,0} = m, m_{i,1} = m)$ as a query and get $c = E(k, m)$. Some queries have different messages in the pairs and some have same messages in the pairs.

Many-time key ciphers is insecure under CPA. For many-time key, the ciphertext $E(k, m)$ is always same for a particular message m . Let the messages are m_0, m_1 . Then adv queries for (m_0, m_0) and gets the ciphertext $c_0 = E(k, m_0)$. Then it queries for (m_0, m_1) and gets $c_b = E(k, m_b)$ for $b = 0, 1$. Now $c_b = c_0 \Rightarrow b = 0 \Rightarrow$ Experiment 0, otherwise Experiment 1. So $Adv[A; E] = 1 \Rightarrow$ insecure. **So for secure ciphers, if the plaintext used multiple times the same secret should encrypt the plaintext differently each time.**

Solution 1: Randomized Encryption: Here a particular plaintext should be encrypted to a set of ciphertexts and that whole set should be decrypted to the particular plaintext. Also encrypting same plaintext twice should give different ciphertexts each time. Roughly, ciphertext size = plaintext size + number of random bit.

Example: Suppose $F : K \times R \rightarrow M$ be secure PRF. $E(k, m) = (r, F(k, r) \oplus m)$, where r is chosen randomly from R . If R set is large enough so that r never repeats with very high probability then $E(k, m) = (r, F(k, r) \oplus m) \approx (r, F(r) \oplus m) \approx (r, r')$. So each time, same plaintext is encrypted into different ciphertext with very high probability. So the encryption is semantically secure.

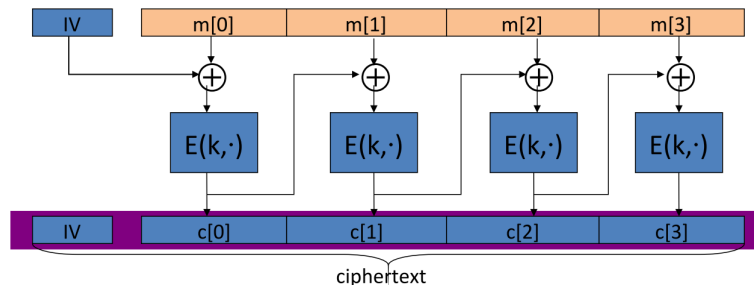
Solution 2: Nonce-based Encryption: It takes two inputs: message m and nonce n and generates ciphertext $E(k, m, n)$ so that no pair (k, n) is used more than once for encryption. Also decryption takes c, n and produces $D(k, c, n) = m$. So if we fix the secret key k , the nonce n should vary message by message. **Example:** Nonce is counter, used when encryptor keeps state from msg to msg. If decryptor has same state then the nonce is not required to send with ciphertext. We can take the nonce to be random $n \leftarrow N$ which never repeats with very high probability $\Rightarrow$ randomized encryption.

CPA security for nonce based encryption: Let the nonce are chosen adversarially. For $i = 1, \dots, q$ the messages $m_{i,0}, m_{i,1}$ and nonces n_i and $c \leftarrow E(k, m_{i,b}, n_i)$. Assume $n_1, \dots, n_q$ are distinct. So E is sem. sec. if for every efficient algo A , $Adv_{nCPA} = |Pr[Exp(0) = 1] - Pr[Exp(1) = 1]|$ is negligible.

Example: Suppose $F : K \times R \rightarrow M$ be sec PRF. Set $r = 0$. Then $E(k, m) = (r + 1, F(k, r + 1) \oplus m)$; $r++$. If R is large enough so that never repeats with high probability.

3.5.7 Block cipher using random IV

Let (E, D) is PRP. Choose random $IV \in X$ and:



So $c[0] = E(k, IV \oplus m[0]) \Rightarrow m[0] = D(k, c[0]) \oplus IV$.

If attacker can predict the IV then the encryption is not CPA-secure, i.e, given $c = E(k, m)$, the attacker can predict the IV . $c_1 = [IV_1, E(k, 0 \oplus IV_1)]$, $m_0 = IV_1 \oplus IV$ and $m_0 \neq m_1$. Now $c = [IV, E(k, IV_1)]$ or $[IV, E(k, m_1 \oplus IV_1)]$. If $c[1] = c_1[1]$ output 0 otherwise 1.

SSL/TLS 1.1 has IV for record #i is the last ciphertext of record #(i - 1).

So we use nonce for block chaining. Key = (k, k_1) . The message is shorter than the ciphertext. So we add a pad so that the sizes are same. For $n > 0$ n byte pad is writing n for n cells. This pad is removed during decryption and the value on last cell tells decryptor to remove how many cells.

3.5.8 Randomized Counter-mode

Let $F : K \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ be secure PRF and IV is a random key for every message. $E(k, IV, m) = IV || m[0] \oplus F(k, IV) || m[1] \oplus F(k, IV + 1) || \dots || m[l] \oplus F(k, IV + l)$. To ensure $F(k, x)$ never repeats we choose $IV = \text{nonce}(64 \text{ bits}) || \text{counter}(64 \text{ bits})$, counter starts at 0 for every message.

3.6 Message Integrity

In this part, we will ensure the message integrity but we will not have the confidentiality, like: protecting public binaries on disk and protecting banner ads on the webpage. The general mechanism is MAC (Message Authentication Code), where a message m is passed through the channel but attacker can't modify the message which is received at the other side properly. It uses **MAC signing algo** which uses $tag = S(k, m)$ and where k is key and m is message and $m || tag$ is passed through the channel. At the receiver end, there is **MAC verification algo** which gives $V(k, m, tag)$ which gives "yes" or "no" depending on that the message is correct or not.

Def1: MAC $I = (S, V)$ is defined over (K, M, T) where $S(k, m) \in T$ and $V(k, m, t)$ gives "yes" or "no". We should get $V(k, m, S(k, m)) = \text{"yes"}$.

If we don't use a secret key, $tag = CRC(m)$ (Cyclic Redundancy Check) and $V(m, tag) = \text{"yes"/"no"}$, attacker can easily change the message and re-compute the CRC. So instead of $m || CRC(m)$, attacker send $m' || CRC(m')$. If we use secret key for calculating tag then attacker can't change the message without knowing the secret key. CRC is designed only to detect random, not malicious errors.

Secure MAC: The attacker has only one possible action: chosen message attack. For $m_1, \dots, m_q$ attacker has $t_i \leftarrow S(k, m_i)$. Attacker's goal (existential forgery) is to produce some new valid message/tag pair (m, t) s.t. $(m, t) \notin \{(m_1, t_1), \dots, (m_q, t_q)\}$. So attacker can't produce a valid tag for a new message also given (m, t) attacker can't produce pair (m, t') , for $t \neq t'$. So we have the game: **Adversary gives messages $m_1, \dots, m_q$ and challenger sends back $t_1, \dots, t_q$ the tags for corresponding messages. Then adversary sends a new pair (m, t) s.t. $(m, t) \notin \{(m_1, t_1), \dots, (m_q, t_q)\}$. If $V(k, m, t) = \text{"yes"}$ then $b = 1 \Rightarrow$ attacker wins otherwise $b = 0 \Rightarrow$ attacker lost.**

Def2: $I = (S, V)$ is secure MAC if for any efficient algo A , $Adv_{MAC}[A, I] = Pr[b = 1]$ is negligible.

(Q) If an attacker is able to find messages $m_0 \neq m_1$ s.t. $S(k, m_0) = S(k, m_1)$ with half of the keys $k \in K$, then is the MAC secure ?

Sol: Let attacker has $m_0 \neq m_1$ s.t. $Pr[S(k, m_0) = S(k, m_1)] = \frac{1}{2}$. Then attacker uses chosen message attack. Attacker sends m_0 and get $t_0 = S(k, m_0)$. Then attacker sends (m_1, t_0) for check. $Pr[V(k, m_1, t_0) = \text{"yes"}] = \frac{1}{2}$, not negligible $\Rightarrow$ MAC is insecure (Valid existential forgery).

(Q) If a MAC is s.t. $S(k, m)$ is always 5 bits long. Is MAC secure ?

Sol: Take any string t of 5 bits long. Then $Pr[V(k, m, t) = \text{"yes"}] = \frac{1}{2^5} = \frac{1}{32}$ not negligible $\Rightarrow$ not secure.

So generally tag length is 64, 96, 128 bits.

Protecting system files: We have files with filenames $F_1, \dots, F_n$. The system generates tags $t_1, \dots, t_n$, where $t_i = S(k, m_i)$, key k is random from the memory. Let some virus tries to change the filenames. User reboots into OS. Secure MAC ensures that all modified files will be detected. If virus tries to swap filenames then that will be detected if the filename is written inside the file.

MAC from PRF: For a PRF $F : K \times X \rightarrow Y$ we define MAC $I = (S, V)$ s.t. $S(k, m) = F(k, m)$ and $V(k, m, t) = \text{"yes"}$ if $t = F(k, m)$ otherwise "no". Secure PRF $\Rightarrow$ Secure MAC. If $Im(F)$ is much less in cardinality (too short length) then MAC is not secure.

Theorem1: If $F : K \times X \rightarrow Y$ is secure PRF and $\frac{1}{|Y|}$ is negligible then I_F is secure MAC. In particular, $Adv[A, I_F] \leq Adv[B, F] + \frac{1}{|Y|}$. So F be secure $\Rightarrow Adv[B, F]$ is negligible and $\frac{1}{|Y|}$ is negligible $\Rightarrow Adv[A, I_F]$ is negligible.
Examples: AES: 16 bytes MAC messages.

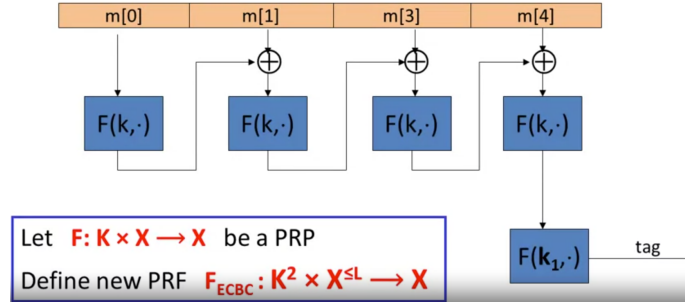
Truncating PRF lemma: $F : K \times X \rightarrow \{0,1\}^n$ be secure PRF then $F_t(k, m) = F(k, m)[1, \dots, t]$ is also secure PRF $\forall 1 \leq t \leq n$.

Similarly truncated MAC into first w is secure MAC as long as $\frac{1}{|w|}$ is negligible.

3.6.1 Constructions

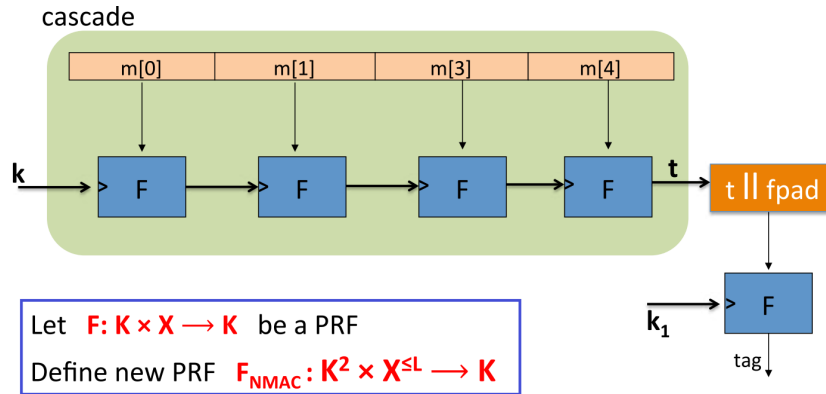
Goal: From PRF of short messages produce PRF for long messages.

Encrypted CBC-MAC:



Let $F : K \times X \rightarrow X$ be PRP. Define $F_{ECBC} : K^2 \times X^{\leq L} \rightarrow X$ where $X^{\leq L} = \bigcup_{i=1}^L X^i$. If the last step was not in ECBC-MAC then it is insecure. Let $S(k, m) = rawCBC(k, m)$ then it is easily broken with 1-chosen message attack. Let adversary has message m . It asks for tag $t = F(k, m)$. Take the new message $(m, t \oplus m)$. Then $rawCBC(k, (m, t \oplus m)) = F(k, F(k, m) \oplus (t \oplus m)) = F(k, t \oplus (t \oplus m)) = F(k, m) = t$. So $(m, t \oplus m)$ has the same tag t . So it becomes insecure.

Nested MAC (NMAC):



Let $F : K \times X \rightarrow K$ be PRF. Define PRF $F_{NMAC} : K^2 \times X^{\leq L} \rightarrow K$. Here k, k_1 are secret keys $\in K$ and $t \in K$. So to take t into X , we do $t || fpad \in X$. Then we do the last step $F(k_1, t || fpad)$ to get the tag. If we don't use the last step using another secret key, then we show that: NMAC isn't secure. Given $cascade(k, m)$ we can produce $cascade(k, m || w)$ for any w , which makes NMAC insecure.

Theorem1: For any $L > 0$, for every efficient q -query PRF adv. A attacking ECBC-MAC, NMAC there exists efficient adv. B s.t. $Adv_{PRF}[A, F_{ECBC}] \leq Adv_{PRF}[B, F] + \frac{2q^2}{|X|}$ and $Adv_{PRF}[A, F_{NMAC}] \leq Adv_{PRF}[B, F] + \frac{q^2}{2|K|}$.
ECBC-MAC and NMAC are secure as long as $q \ll \sqrt{|X|}$ and $q \ll \sqrt{|K|}$ (2^{64} for AES)

Extension Property: If the used PRF is also a PRP then both ECBC-MAC and NMAC will have: $F(k, x) = F(k, y) \Rightarrow F(k, x||w) = F(k, y||w) \forall x, y, w$.

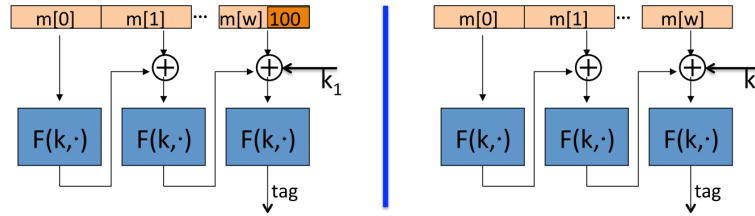
Generic attack: Let $F : K \times X \rightarrow Y$ be PRF which has extension property. Then we have a generic attack using existential forgery. Asks for $\sqrt{|Y|}$ message queries of random messages and get (m_i, t_i) . Then by Birthday paradox, we get $u \neq v$ s.t. $t_u = t_v$ (with very high probability). Then choose w and query for $t = F(k, m_u||w)$. Then we are able to produce an existential forgery $t = F(k, m_v||w)$.

Uses: ECBC-MAC is used in AES-based MAC (CCM encryption mode). NIST standard called CMAC. NMAC is the basis of popular MAC called HMAC.

MAC padding: For security of MAC-padding we need the fn pad to be one-one, i.e., $m_0 \neq m_1 \Rightarrow \text{pad}(m_0) \neq \text{pad}(m_1)$ and also fn pad is invertible.

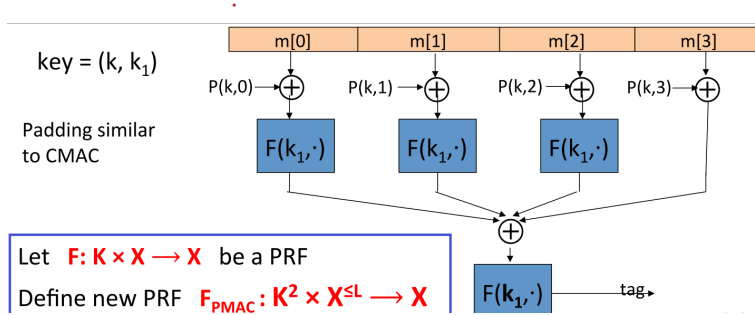
ISO: Pad with 100... i.e., $m[0]m[1] \dots \Rightarrow m[0]m[1] \dots || 100 \dots$. The rightmost 1 denotes the beginning of the pad. Add new dummy block if needed.

CMAC:



There is no final encryption step and no dummy block. There are 3 keys k, k_1, k_2 .

Parallel MAC:



Here $F : K \times X \rightarrow X$ is a PRF. Define new PRF $F_{PMAC} : K^2 \times X^{\leq L} \rightarrow X$. If there is no fn p then if we swap $m[i]$ and $m[j]$ the tag remains same. So it creates existential forgery. The fn p takes a secret key k and block number n as input which is easily computable.

Theorem3: $Adv_{PRF}[A, F_{PMAC}] \leq Adv_{PRF}[B, F] + \frac{2q^2L^2}{|X|}$, secure upto $qL \ll \sqrt{|X|}$.

(Q) Given a tag of a message m , how to get tag for the message where $m[1]$ is replaced by $m'[1]$.

Sol: We have tag t for m . We do $F^{-1}(k_1, t)$. Now we can cancel the component of $m[1]$ by xor $m[1]$ component with $F^{-1}(k_1, t)$ and add the component of $m'[1]$, i.e., $F^{-1}(k_1, t) \oplus F(k_1, m[1] \oplus P(k, 1)) \oplus F(k_1, m'[1] \oplus P(k, 1))$ which is the tag for the new message.

One-time MAC: For a MAC $I = (S, V)$ and adversary A define a game: A gives a message $m_1 \in M$ to challenger and challenger returns $t_1 = S(k, m_1)$. Then A sends the pair of message/tag (m, t) . If $V(k, m, t) = \text{"yes"}$ then $b = 1$ otherwise $b = 0$.

Def: A MAC is secure if for all efficient adversary A $Adv_{MAC}[A, I] = Pr[b = 1]$ is negligible.

Construction: Let q be a large prime (e.g. $q = 2^{128} + 51$). Let $key = (k, a) \in \{1, \dots, q\}^2$, 2 random integer and $msg = (m[1], \dots, m[L])$ where each block is 128 bit int. $S(key, msg) = P_{msg}(k) + a \pmod{q}$, where $P_{msg}(x) = m[L]x^L + \dots + m[1]x$.

Fact: Given $S(key, msg_1)$ adv. has no information about $S(key, msg_2)$.

Carter-Wegman MAC (many-time MAC): Let (S, V) be secure one-time MAC over $(K, M, \{0, 1\}^n)$. Let $F : K_F \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ be secure PRF. Then $CW((k_1, k_2), m) = (r, F(k_1, r) \oplus S(k_2, m))$, where random $r \leftarrow \{0, 1\}^n$. This is a randomized MAC.

Theorem4: (S, V) secure one-time MAC and F secure PRF $\Rightarrow CW$ is secure MAC outputting tag $\in \{0, 1\}^{2n}$.

Verifying algo for CW MAC: If $V(k_2, m, tag)$ is the algo for verifying the integrity of the message then **Run** $V(k_2, m, F(k_1, r) \oplus t)$ and get the output. Because $tag = (r, t)$, where $t = F(k_1, r) \oplus S(k_2, m)$, then $F(k_1, r) \oplus t = S(k_2, m)$. So $V(k_2, m, S(k_2, m))$ will give the correct output.

3.6.2 Collision Resistance

Def1: Let $H : M \rightarrow T$ is a Hash function s.t. $|M| \gg |T|$. A collision for H is a pair $m_0, m_1 \in M$ s.t. $H(m_0) = H(m_1)$ and $m_0 \neq m_1$.

Def2: A fn H is **collision resistant** if for all (explicit) efficient algo A , $Adv_{CR}[A, H] = Pr[A \text{ outputs collision for } H]$ is negligible. e.g. SHA-256 (outputs 256 bits).

MAC from collision resistance: Let $I = (S, V)$ be a MAC over (K, M, T) . Let $H : M^{big} \rightarrow M$ be a Hash fn. Define $I^{big} = (S^{big}, V^{big})$ over (K, M^{big}, T) s.t. $S^{big} = S(k, H(m))$ and $V^{big} = V(k, H(m), t)$.

Theorem1: I be secure MAC and H is collision resistant $\Rightarrow I^{big}$ is secure MAC.

Pf: Given some message/tag pairs $\{(m_1, t_1), \dots, (m_q, t_q)\}$: $Adv_{MAC}[A, I] = Pr[V(k, m', t') = \text{"yes"}] = Pr[\{H(m') = H(m_i)\} \cup \{\text{predict tag}\}] \leq Pr[H(m') = H(m_i)] + Pr[\text{predict tag}] = \text{negligible} + \text{negligible} = \text{negligible}$. If H is not collision resistant then we can find $m' \neq m$ s.t. $H(m) = H(m')$. Then by chosen 1-message attack we can produce existential forgery by $(m', S(k, m))$.

Example: $S(k, m) = AES_{2 \text{ block } CBC}(k, SHA - 256(m))$ is secure MAC.

Generic attack on C.R. functions: Let $H : M \rightarrow \{0, 1\}^n$ s.t. $|M| \gg 2^n$. Generic algo to find a collision in time $O(2^{\frac{n}{2}})$ hashes. The algo is the following:

1. Choose random $2^{\frac{n}{2}}$ messages $m_1, \dots$ (distinct w.h.p).
2. Calculate $t_i = H(m_i)$.
3. Look for collision ($t_i = t_j$). if not found then go back to step 1.

Quantum algo takes $O(2^{\frac{n}{3}})$ hashes to find a collision.

The Birthday Paradox: Let $r_1, \dots, r_n \in \{1, \dots, B\}$ be indep identically distributed integers. When $n = 1.2 \times \sqrt{B}$ then $Pr[\exists i \neq j : r_i = r_j] \geq \frac{1}{2}$.

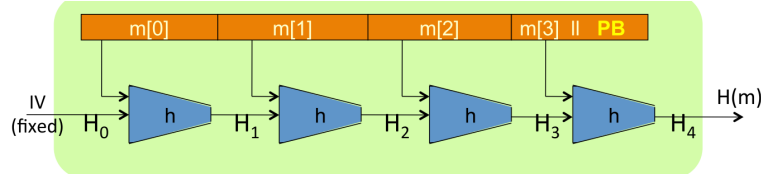
Pf: For uniform indep $r_1, \dots, r_n$: $Pr[\exists i \neq j : r_i = r_j] = 1 - Pr[\forall i \neq j : r_i \neq r_j] = 1 - \frac{B-1}{B} \cdot \frac{B-2}{B} \cdot \dots \cdot \frac{B-n+1}{B} = 1 - \prod_{i=1}^{n-1} (1 - \frac{i}{B}) \geq 1 - \prod_{i=1}^{n-1} e^{-\frac{i}{B}} = 1 - e^{-\frac{1}{B} \sum_{i=1}^{n-1} i} \geq 1 - e^{-\frac{n^2}{2B}} = 1 - e^{-0.72} = 0.53 \geq \frac{1}{2}$.

Using the Birthday Paradox, expected # of iterations required for the generic attack algo = 2. Running time = $O(2^{\frac{n}{2}})$ and Space complexity = $O(2^{\frac{n}{2}})$.

Examples: SHA-256: Generic attack time = 2^{128} and SHA-512: Generic attack time = 2^{256} .

Now our goal is to create CR fns for long messages when CR fns for short messages is given.

Merkle Damgard iterated construction:

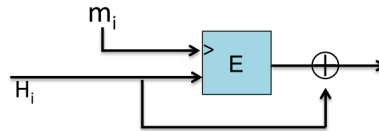


Here we have $h : T \times X \rightarrow T$ (compression fn) is collision resistant fn for shorter messages. Then we break long message m into blocks $m[0], \dots$. At the end we add a padding block (PB) with m . Now a fixed value IV is fed into fn h with another input $m[0]$. Then output of that is fed into h with input $m[1]$ and so on. When the last block is fed into h then the output of that h is the output $H(m)$ of the constructed CR fn H s.t. $H : X^{\leq L} \rightarrow T$. The padding block is $100\dots 0 || \text{msg len encoding (64 bits)}$. If no space is remaining for PB then add another block.

Theorem2: h is CR fn $\Rightarrow H$ is CR fn.

Pf: Suppose there is a collision in H , $H(m) = H(m')$. Let $H_0, H_1, \dots$ are the chaining variables at each step of applying h . Then for m and m' : $IV = H_0, H_1, \dots, H_{t+1} = H(m)$ and $IV = H'_0, H'_1, \dots, H'_{r+1} = H(m') = H(m)$. Now $H(m) = H_{t+1} = h(H_t, m_t || PB) = H(m') = H'_{r+1} = h(H'_r, m'_r || PB')$ which produces a collision in fn h , if $(H_t, m_t || PB) \neq (H'_r, m'_r || PB')$. So collision of H implies collision of h . Then if h is collision resistant then so H is. If $(H_t, m_t || PB) = (H'_r, m'_r || PB')$, then $H_t = H'_r$ and $m_t || PB = m'_r || PB'$. Then we do the same argument as for $H_{t+1} = H'_{r+1}$. If we come to the beginning of the constructor then we get $m = m'$, which tells us it is not a collision in H .

Constructing Compression function: Our goal is to create compression fn $h : T \times X \rightarrow T$. Let $E : K \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ be a block cipher. The **Davies-Meyer compression function** is: $h(H, m) = E(m, H) \oplus H$.



Theorem3: E is ideal cipher (collection of $|K|$ random permutations) $\Rightarrow$ Finding a collision takes $O(2^{\frac{n}{2}})$ evaluations of (E, D) .

Suppose we define: $h(H, m) = E(m, H)$. Then for m' define: $H' = D(m', E(m, H))$. Then $E(m', H') = E(m', D(m', E(m, H))) = E(m, H)$ which is a collision.

Other constructions: Let $E : \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}^n$.

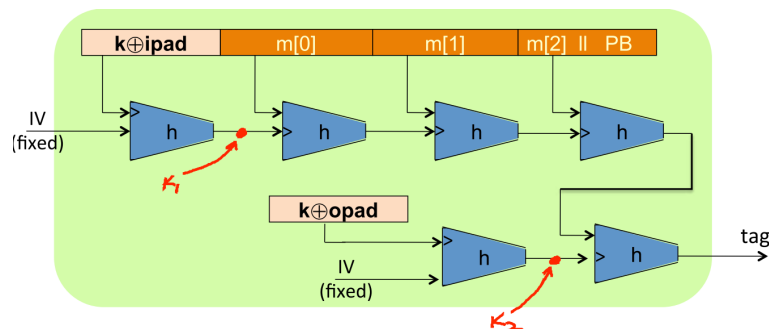
Miyaguchi-Preneel: $h(H, m) = E(m, H) \oplus H \oplus m$ or $h(H, m) = E(H \oplus m, m) \oplus m$, 12 variants like these. Other natural variants like: $h(H, m) = E(m, H) \oplus m$ are insecure. (Take $H' = D(m', h(m, H) \oplus m)$).

Provable compression fn: Choose a random 2000 bits prime p and random $1 \leq u, v \leq p$. For $m, H \in \{0, \dots, p-1\}$ define $h(H, m) = u^H \cdot v^m \mod p$.
Fact: Finding the collision in this model is as hard as solving "discrete-log" modulo p .

3.6.3 MAC from Hash functions

MAC from Merkle-Damgard Hash fn: $H : X^{\leq L} \rightarrow T$ a C.R. Merkle-Damgard Hash fn. Then the definition $S(k, m) = H(k||m)$ is insecure because, given $H(k||m)$ we can compute $H(k||m||PB||w)$ for any w .

HMAC: Most widely used MAC on the internet, e.g. SHA-256. Define:
 $H(k, m) = H(k \oplus opad || H(k \oplus ipad || m))$, where $ipad$ = inner pad and $opad$ = outer pad.



Similar to the NMAC PRF. Only difference is: in NMAC PRF, keys k_1, k_2 are random, but in HMAC k_1, k_2 is dependent on the output of fn h . Security bounds are also same as NMAC PRF (required $\frac{q^2}{|T|}$ to be negligible). In TLS: must support HMAC-SHA1-96.

Verification Timing Attack: Suppose we have a message m and we want to find its tag produced in a server with secret key k . There we use a timing attack. To compute the tag we do the following:

1. Query server with a random tag.
2. Loop over all possible leftmost byte and query server. Every time we generate the tag and pass message/tag pair and the server accept or reject. Then we stop the loop till the verification takes a little longer time than others in the server. Then that byte is the correct leftmost byte of the required tag.
3. Repeat for all tag bytes until the valid tag is found.

Example: The fn Verify in Keyczar crypto library:

```
def Verify(key,msg,sig-bytes):
    return HMAC(key,msg)==sig-bytes
```

Here '==' is implemented as a byte-by-byte comparison. So for this implementation, the tag can be found by Verification Timing Attack.

Defense#1: We have to change the implementation of the string comparator, s.t. it takes always the same time.

```
return false if sig-bytes has wrong length
result=0
for x, y in zip(HMAC(key,msg),sig-bytes):
    result |= ord(x)ord(y)
return result==0
```

This approach can be difficult to ensure due to optimizing compiler.

Defense#2: Making string comparator which always takes same time.

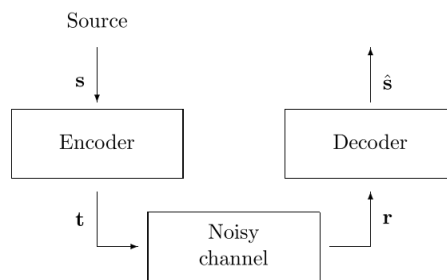
```
def Verify(key,msg,sig-bytes):
    mac=HMAC(key,msg)
    return HMAC(key,mac)==HMAC(key,sig-bytes)
```

It is comparing the strings after computing the fn HMAC every time. So it takes always same time.

Chapter 4

Information Theory

We have some messages and we want to do send the messages through a noisy channel such that in between communication no one can get the actual message and only at the end the receiver can successfully decode the message. So our structure would be like:



Binary Symmetric Channel: Let $0 \leq f \leq 1$. Define $Pr[x = 0|y = 0] = 1 - f$, $Pr[x = 1|y = 0] = f$, $Pr[y = 0|x = 0] = 1 - f$, $Pr[y = 1|x = 0] = f$. If we want to write a GB per day for 10 years then: number of bits = $10 \cdot 365 \cdot 8 \times 10^9 = 2.9 \times 10^{13}$. To get $f = 0.1$ we require bit-error probability of the order 10^{-15} . Entropy of the system: $H(f) = f \cdot \log_2 \frac{1}{f} + (1 - f) \cdot \log_2 \frac{1}{1-f}$. The capacity of the channel $C(f) = 1 - H(f)$. If $f = 0.1$ then $C(f) \approx 0.53$.

Trick: For a problem of finding the least number of steps for finding a result, Shannon's information content tells to try to get the steps which can give us highest information (maximum entropy).

4.1 Data Compression

Def: An ensemble X is a triple (x, A_X, P_X) where x is outcome of r.v. X , A_X is the set of possible outcomes of X , P_X is the set of probabilities of the outcomes of X .

Game 1 (Sixty three): Take a number $\in \{0, 1, \dots, 63\}$. Try to do least number questions to guess the number correctly. Do the questions $(X \equiv \text{mod } 32) \geq 16$, $(X \equiv \text{mod } 16) \geq 8, \dots$, $(X \equiv \text{mod } 2) \geq 1$. All of these questions if YES then 1 otherwise 0. If the string we get is 101010 then $X = 42$ which is 101010 in binary. $H(Q_i) = \log_2 \frac{1}{P(Q_i)} = \log_2 \frac{1}{2} = 1$ bit. So total number of bits = 6 bits. Similarly is N possible outcomes then number of bits long much is required for each number's unique name is: $\lceil \log_2 N \rceil$.

Game 2 (Bent-coin Lottery): There are 1000 bits lottery tickets each of price Rs.1. Wining prize: Rs.1000000000. There is a coin with $f = 0.1$ which is tossed 1000 times. The string of 1000 bits got from the tosses is the wining number. How many tickets we have to buy to ensure 99% wining chance (Answer of the form 2^k).

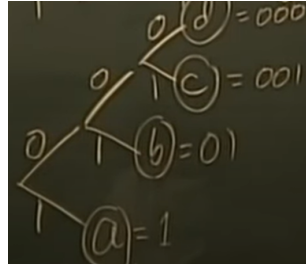
$E(X) = Nf = 100$ and $Var(X) = Nf(1-f) = 90 \Rightarrow \sigma \approx 10$. Then if we buy the all tickets with 0 1s, 1 1s, ..., $(100+20 = 120)$ 1s then we the chances of wining = 95% (middle) + 2.5% (left side) = 97.5%. Number of tickets to buy = $1 + 1000 + \binom{1000}{2} + \dots + \binom{1000}{120}$. To achieve 99% we add some more terms. Number of tickets to buy = $1 + 1000 + \binom{1000}{2} + \dots + \binom{1000}{123}$. We have a fact: $\log_2 \binom{N}{r} = NH_2(\frac{r}{N})$. Using this fact in general: number of tickets to buy = $1 + \dots + \binom{N}{fN + [Nf(1-f)]^{\frac{1}{2.3}}} \approx 2^{NH_2(f)(\sqrt{N} + \dots)}$, where $H_2(f) = f \log_2 \frac{1}{f} + (1-f) \log_2 \frac{1}{1-f}$.

Def: The raw bit content of X is $H_0(X) = \log_2 |A_X|$.

Source Coding Theorem: N outcomes from a source X can be compressed into roughly $NH(X)$ bits, where $H(X)$ is the entropy of the source X .

4.1.1 Symbol codes

We define a function C from set of alphabets to set of binary strings. Expected length of the symbol code C : $L(C, X) = \sum_i p_i \cdot l_i$ where l_i is the length and p_i is the probability of the symbol i . We want the symbol coding s.t. it is easily and uniquely decodable, i.e., for $x \neq y$, $C(x) \neq C(y)$ and also small $L(C, X)$. **Example:** A set with 4 symbols can be coded as: $\{1, 01, 001, 000\}$, where give lesser length strings to higher probability symbols. We can check this gives the symbol code with smallest $L(C, X)$. It is also called prefix code so that no other codeword is prefix of any other. Prefix code we can get from binary tree structure.



If we have 0 in the prefix code then only 3 length strings which can be included in the code are: 100, 110, 101, 111. For length l , there are 2^l codewords. Set cost of each l length string to be 2^{-l} . So total spending = $\sum_i 2^{-l_i}$. Define total budget to be 1. Then $\sum_i 2^{-l_i} \leq 1$.

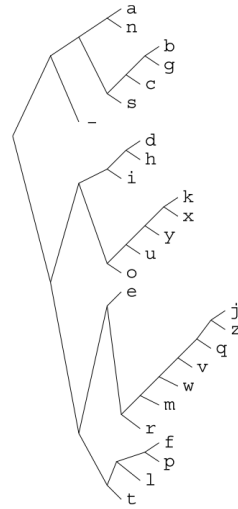
Kraft's inequality: If $C(X)$ is uniq. decod. code then it must satisfy: $\sum_i 2^{-l_i} \leq 1$.

Def: The code satisfying Kraft's inequality is called complete code.

Define $\tilde{l}_i = \log_2 \frac{1}{p_i}$. Let $Z = \sum_i 2^{-l_i}$. For any uniq. decod. code $Z \leq 1$. Define $q_i = \frac{2^{-l_i}}{Z} \Rightarrow l_i = \log_2 \frac{1}{q_i} - \log_2 Z$. $L(C, X) = \sum_i p_i (\log_2 \frac{1}{q_i} - \log_2 Z) = \sum_i p_i \log_2 \frac{1}{p_i} + \sum_i p_i \log_2 \frac{p_i}{q_i} - \log_2 Z \sum_i p_i = H(X) + D(p||q) - \log_2 Z$. $Z \leq 1 \Rightarrow -\log_2 Z \geq 0$ and by Gibb's inequality $D(p||q) \geq 0$, only equality holds if $p = q$. So $L(C, X) \geq H(X)$ with equality iff $l_i = \tilde{l}_i$ and complete code. We get $l_i = \lceil \log_2 \frac{1}{p_i} \rceil \Rightarrow L(C, X) < \sum_i p_i (\log_2 \frac{1}{p_i} + 1) = H(X) + 1$. So $H(X) \leq L(C, X) < H(X) + 1$. So there exists an optimal symbol coding for a given (x, A_X, P_X) .

Optimal Symbol Coding: Huffman algorithm gives the optimal symbol coding which says that bulid a binary tree starting from the leaves (smaller probabilities) and combine the symbols with lowest probabilities. For example: let the probabilities are 0.4, 0.14, 0.12, 0.13, 0.11, 0.1. Combine 0.1, 0.11=0.21; 0.12, 0.13=0.25; 0.14, 0.21=0.35; 0.25, 0.35=0.6; 0.4, 0.6=1. Now starting from 1 gives 0 for lower probability edge and 1 for higher probability edge. So we get the codewords for each symbol. The following is the optimal coding for English alphabets using Huffman algo.

a_i	p_i	$\log_2 \frac{1}{p_i}$	l_i	$c(a_i)$
a	0.0575	4.1	4	0000
b	0.0128	6.3	6	001000
c	0.0263	5.2	5	00101
d	0.0285	5.1	5	10000
e	0.0913	3.5	4	1100
f	0.0173	5.9	6	111000
g	0.0133	6.2	6	001001
h	0.0313	5.0	5	10001
i	0.0599	4.1	4	1001
j	0.0006	10.7	10	1101000000
k	0.0084	6.9	7	1010000
l	0.0335	4.9	5	11101
m	0.0235	5.4	6	110101
n	0.0596	4.1	4	0001
o	0.0689	3.9	4	1011
p	0.0192	5.7	6	111001
q	0.0008	10.3	9	110100001
r	0.0508	4.3	5	11011
s	0.0567	4.1	4	0011
t	0.0706	3.8	4	1111
u	0.0334	4.9	5	10101
v	0.0069	7.2	8	11010001
w	0.0119	6.4	7	1101001
x	0.0073	7.1	7	1010001
y	0.0164	5.9	6	101001
z	0.0007	10.4	10	1101000001
-	0.1928	2.4	2	01



Problem with Huffman coding: Huffman algo requires many time 1 extra bit which makes it inefficient. The overhead $L(C, X) - H(X)$ may dominate the encoded file length. It is context dependent so any predictive guess makes huge difference in the context. Huffman algo can't handle the change of ensemble. So when it changes may be probabilities varies we have to re-compute using the algo.

4.1.2 Arithmetic Codes

We can view the binary strings by defining each string as an interval over the real line. Example: $01 \equiv [0.25, 0.5]$, $100 \equiv [0.5, 0.625]$. This definition comes from the binary tree structure: for 01, 0 $\equiv [0, 0.5]$ then 1 so divide the interval $[0, 0.5]$ into two halves and take the 2nd half so $[0.25, 0.5]$. Similarly for 100, first 1 defines $[0.5, 1]$ then divide into 2 halves twice and take the leftmost part. A 1Mb file has 8×10^6 bits which defines a real number with ≈ 2 million decimal places. We can encode any binary string $x_1x_2...x_n$ like this. For bent coin with $P(a) = 0.9$, $P(b) = 0.1$: $H(0.9, 0.1) \approx 0.5$.

Given an ensemble, we subdivide interval $[0, 1]$ by $P(x_1)$, then subdivide by $P(x_2|x_1)$ again by $P(x_3|x_1, x_2)$ and so on. So multiplying the ratios we get the joint distribution $P(x_1, x_2, ..., x_n)$. In the worst case, we can prove that: the product of the ratios 4 times $P(x_1, x_2, ..., x_n)$. By Shannon's information content, length of the binary string:

$$l(x_1...x_n) \leq \log_2 \frac{1}{p(x_1...x_n)} + 2$$

. We have to only compute n, I = alphabet size and the conditional probabilities for arithmetic coding of the given ensemble. This method is compatible with any source alphabet and any transmitting alphabet. Arithmetic coding is used in **JBIG**, **PPM**, **DJVV**.

Uses of arithmetic coding is in efficient writing (e.g. Dasher) where compression: text $\rightarrow$ bit-string and writing: Gesture $\rightarrow$ text, efficiently generating random numbers.

4.2 Noisy Channel Coding

Def 1: Given joint distribution of 2 random variables X, Y the **Marginal entropy** $H(X, Y) = \sum_{x,y} P(x, y) \log_2 \frac{1}{P(x, y)}$.

Def 2: The **Conditional entropy** $H(X|Y) = \sum_y P(y) H(X|y)$.

Thm 1: $H(X) \geq H(X|Y)$, $H(X) + H(Y|X) = H(Y) + H(X|Y) = H(X, Y)$ and $H(X) - H(X|Y) = I(X; Y) = H(Y) - H(Y|X)$, $I(X; Y)$ is the **Mutual Information**.

Bayes's Thm: Posterior $P(H|Data) = \frac{Likelihood(P(Data|H)) \cdot Prior(P(H))}{P(Data)}$

A channel Q defines conditional probability $P(y|x)$. Given joint distribution we have $P(x, y) = P(x) \cdot P(y|x)$, with which we can do inference.

Symmetric Channel: Binary symmetric channel with $Pr[Y = 1|X = 0] = f = Pr[Y = 0|X = 1]$ and $Pr[Y = 0|X = 0] = Pr[Y = 1|X = 1] = 1 - f$. **Binary erasure channel** with $Pr[Y = 0|X = 0] = Pr[Y = 1|X = 1] = 1 - f$ and $Pr[Z = u|X = 0] = f = Pr[Z = u|X = 1]$, where u is unknown symbol.

Asymmetric Channel: Z-channel with $Pr[Y = 0|X = 1] = f$ and $Pr[Y = 1|X = 1] = 1 - f$.

So we can define a matrix for conditional probability $Q_{j|i} = Pr[Y = b_j|X = a_i]$.

Mutual information for BSC: $Q[1][1] = Q[2][2] = 1 - f$, $Q[1][2] = Q[2][1] = f$. Let $f = 0.1$. So input distribution $P_X = \{p_0 = 0.9, p_1 = 0.1\}$. $I(X; Y) = H(Y) - H(Y|X) = H_2(0.18) - H_2(0.1) = 0.21$ bits.

Def 3: Capacity of a channel $C(Q) = \max_{P_X} I(X; Y)$.

Example: For BSC with $f = 0.1$ and input distribution $p_1 \rightarrow (p_1, 1 - p_1)$,

$I(X; Y) = H_2(p_1(1 - f) + (1 - p_1)f) - H_2(f)$ maximum value $1 - H_2(f)$ attains at $p_1 = \frac{1}{2}$. So $C_{BSC}(f) = 1 - H_2(f)$. For $f = 0.1$, $C_{BSC} \approx 0.53$.

Ternary confusion channel: Input distribution $P_X = \{\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\}$, where $a \rightarrow M, c \rightarrow N$ and $b \rightarrow \frac{1}{2}M, b \rightarrow \frac{1}{2}N$. So $Q[1][1] = Q[2][3] = 1$, $Q[1][2] = Q[2][2] = \frac{1}{2}$, $Q[1][3] = Q[2][1] = 0$. $Pr[X|Y = M] = \{\frac{2}{3}, \frac{1}{3}, 0\}$. Also $I(X; Y) = H(Y) - H(Y|X) = 1 - (\frac{1}{3} \cdot 0 + \frac{1}{3} \cdot \log_2 \frac{1}{2} + \frac{1}{3} \cdot 0) = \frac{2}{3}$.

Noisy Typewriter: Suppose $Pr[Y = A|X = B] = Pr[Y = B|X = B] = Pr[Y = C|X = B] = \frac{1}{3}$ and so on for other alphabets. Capacity = $\max_{P_X} I(X; Y) = \max_{P_X} H(Y) - H(Y|X) = \max_{P_X} H(Y) - \log_2 3$. For maximum of $H(Y)$ we do uniform distribution for all 27 alphabets $\{A, \dots, Z, -\}$. So $H(Y) = \log_2 27$. So $I(X; Y) = \log_2 9$ bits and optimal input distribution P_X is uniform distribution or $P_X = \{0, \frac{1}{9}, 0, 0, \frac{1}{9}, 0, \dots\}$.

Extended channel: In general, if there are N symbols and if we take P_X to be s.t. each symbol has $NH_2(Y|X)$ options of symbols with uniform probability, so $2^{NH_2(Y|X)}$ and total $2^{NH_2(Y)}$. Then number of almost certainly non-confusable inputs = $\frac{2^{NH_2(Y)}}{2^{NH_2(Y|X)}} = 2^{NI(X; Y)} \leq 2^{NC}$, NC bits per N channel at a time.

Noisy-channel Coding Theorem: Given a noisy channel of capacity C and the information is transmitted at a rate R , where $R < C$ then for every $\epsilon \exists$ codes for which the probability of error at the receiver is $< \epsilon$.

Capacity of Binary Erasure Channel: $Pr[X = i] = p_i$, $Pr[Y = 0|X = 0] = 1 - f = Pr[Y = 1|X = 1]$ $I(X; Y) = H(X) - H(X|Y) = H_2(p_0) - fH_2(p_0) = (1 - f)H_2(p_0)$ maximum attains at $p_0 = \frac{1}{2} \Rightarrow C = (1 - f)H_2(\frac{1}{2}) = 1 - f$.

4.2.1 7,4 Hamming Code

For $M = 3$ and $N = 7$, the **parity check matrix** of 7,4 Hamming code is:
$$\begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

Valid transmissions t satisfy $Ht = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \pmod{2}$. In a channel, **received signal** $r = t + n \pmod{2}$,

t is transmission and n is noise. **Syndrome** $Z = Hr = H(t + n) = Ht + Hn = Hn \pmod{2}$. Looking at the column matrix Z , we can detect which column represents Z and from that we can get the smallest noise $\hat{n} \equiv n \pmod{2}$, which bits are flipped.

Let H is $M \times N$ matrix, $K = N - M$. Assume H has no redundant constraints. Then the **rate** of code $= \frac{K}{N}$.

Q1: If x is a fixed non-zero binary vector and h is random $(\frac{1}{2}, \frac{1}{2})$ binary vector of same length then probability $h^t x = 0 \pmod{2}$ is $\frac{1}{2}$. (x is fixed and $h = (h_1, \dots)^t$ where h_i s are tosses of a fair coin. $h^t x$ gives a constraint which gives possible values of h_i , so by symmetry $Pr[h^t x = 0] = \frac{1}{2}$)

Q2: Given a fixed non-zero binary vector x of length N and a random matrix H of $M \times N$, the probability of $Hx = 0 \pmod{2}$ is $(\frac{1}{2})^M$. (Each row of H is independent of each other, each row h_i : $Pr[h_i x = 0] = \frac{1}{2}$ and there are M rows, so $(\frac{1}{2})^M$)

The average probability of error: $\langle P(H) \rangle_H = \sum_{n \in \text{bag}} P(n) \sum_{\tilde{n} \in \text{bag}, \tilde{n} \neq n} (\frac{1}{2})^M \leq 1 \cdot 2^{NH_2(f)} \cdot (\frac{1}{2})^M$

$\Rightarrow \langle P(H) \rangle_H \leq P_I + \frac{1}{2^{M-NH_2(f)}}$, which vanishes when $M \gg NH_2(f) \Rightarrow \frac{M}{N} \gg H_2(f) \Rightarrow 1 - \frac{M}{N} \ll 1 - H_2(f) \Rightarrow R \ll C$, since $R = \frac{N-M}{M}$ and $C = 1 - H_2(f)$.

4.3 Bayesian Inference

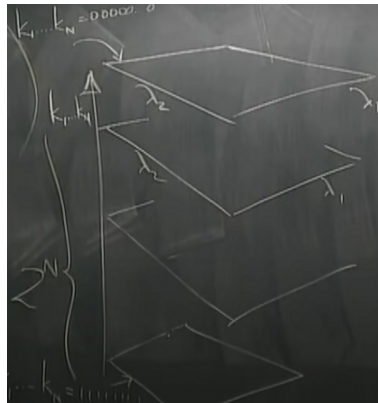
Probability distribution: $\mu(a, b, \lambda) = \langle x \rangle = \frac{\int_a^b e^{-\frac{x}{\lambda}} dx}{\int_a^b e^{-\frac{x}{\lambda}} dx}$

$Pr[x|\lambda, H] = \frac{e^{-\frac{x}{\lambda}}}{Z}$, $Z = \int_a^b e^{-\frac{x}{\lambda}} dx = \lambda(e^{-\frac{a}{\lambda}} - e^{-\frac{b}{\lambda}}) \Rightarrow Pr[\{x\}|\lambda, H] = \prod_1^N \frac{e^{-\frac{x_n}{\lambda}}}{Z(\lambda, a, b)}$

So $Pr[\lambda|\{x\}, H] = \frac{Pr[\{x\}|\lambda, H] \cdot Pr[\lambda|H]}{Pr[\{x\}|H]}$ where $Pr[\{x\}|\lambda, H] = \frac{e^{-\sum_1^N \frac{x_n}{\lambda}}}{Z(\lambda, a, b)^N}$. If λ_i are exponential and

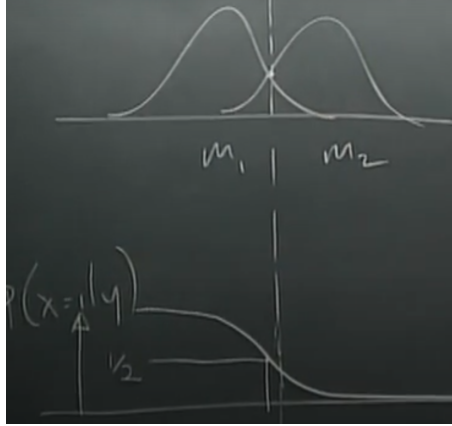
$\pi_i > 0$ are weights and $\pi_1 + \pi_2 = 1$ then $Pr[x|\lambda_1, \lambda_2, \pi_1, \pi_2, H] = (\pi_1 \cdot \frac{e^{-\frac{x}{\lambda_1}}}{\lambda_1} + \pi_2 \cdot \frac{e^{-\frac{x}{\lambda_2}}}{\lambda_2})$ for $x \in (a, b)$ otherwise 0. Similarly, $Pr[\lambda_1, \lambda_2|\{x\}, H] = \frac{Pr[\{x\}|\lambda_1, \lambda_2, H] \cdot Pr[\lambda_1, \lambda_2|H]}{Pr[\{x\}|H]}$

Model comparison: $Pr[H_1|\{x\}, I] = \frac{Pr[\{x\}|H_1, I] \cdot Pr[H_1|I]}{Pr[\{x\}|I]}$ and $Pr[H_2|\{x\}, I] = \frac{Pr[\{x\}|H_2, I] \cdot Pr[H_2|I]}{Pr[\{x\}|I]}$
 $Pr[\{x\}, \{k\}|H, \lambda_1, \lambda_2] = \prod_n Pr[k_n|H, \lambda_1, \lambda_2] \cdot Pr[x_n|k_n, H, \lambda_1, \lambda_2]$. We can visualize the quantity $Pr[\{x\}_1^N|\lambda_1, \lambda_2, \{k_n\}]$ as:



4.3.1 Clustering

Gaussian channel: This channel has inputs $x \in \{1, 2\}$ and real output y with $P[x = 1] = \pi_1$ and $P[y|x] = \frac{e^{-\frac{(y-m_x)^2}{2\sigma^2}}}{\sqrt{2\pi\sigma^2}} = K(x, y)$. So $P[x = 1|y] = \frac{\pi_1 K(x, y)}{P[y|x=1]\pi_1 + P[y|x=2]\pi_2} = \frac{1}{1 + e^{-a(y)}}$ where $a(y) = \frac{(y-m_2)^2 - (y-m_1)^2}{2\sigma^2} + \log \frac{\pi_1}{\pi_2} = \frac{1}{\sigma^2}(y - \frac{m_1+m_2}{2})(m_1 - m_2) + \log \frac{\pi_1}{\pi_2}$



Steps for clustering (Kmeans Algorithm): Spherical K gaussian with same σ all over the sphere and $\pi_1 = \pi_2 = \dots = \pi_k$. Apply this algo till there is no more updates.

1. Initialize K , i.e., give the means for the points in the graph $\{m^{(k)}\}_{k=1}^K$ at random.
2. Define a distance $d(m^{(k)}, x^{(n)}) = \frac{1}{2}(m^{(k)} - x^{(n)})^2$ then assign $x^{(n)}$ to the nearest $m^{(k)}$. We can define 'Responsibility' $r_k^{(n)} = 1$ if $m^{(k)}$ is the closest mean to $x^{(n)}$ otherwise 0, or k minimizes $d(m^{(k)}, x^{(n)})$.
3. Update $m^{(k)} = \frac{\sum_{n=1}^N r_k^{(n)} x^{(n)}}{\sum_{n=1}^N r_k^{(n)}}$, where N = number of clusters.

Kmeans is a map algo which adjusts $\{x_n\}, \{m_k\}$ to maximize the posterior probability.

Soft Kmeans Algorithm:

1. Initialize K , i.e., give the means for the points in the graph $\{m^{(k)}\}_{k=1}^K$ at random and set a 'stiffness' parameter β .
2. Assign 'softly' $r_k^{(n)} = \frac{e^{-\beta d(m^{(k)}, x^{(n)})}}{\sum_{k'} e^{-\beta d(m^{(k')}, x^{(n)})}}$
3. Update $m^{(k)} = \frac{\sum_{n=1}^N r_k^{(n)} x^{(n)}}{\sum_{n=1}^N r_k^{(n)}}$, where N = number of clusters.

This algo gives different output (different clusters) if we change the value of β .

Soft assignment: The previous algo gives the clusters of circle shape with certain radius, which sometimes doesn't give good clusters. So we can add some softness to the model. Instead of β , use variables $\sigma_i^{(k)}$. Also we update $\sigma_i^{(k)} = \frac{\sum r_k^{(n)} (x_i^{(n)} - m_i^{(k)})^2}{\sum r_k^{(n)}}$. We can add more variables $\{\pi_k\}$ and

update $\pi_k = \frac{\sum r_k^{(n)}}{N}$.

Example: The posterior probability θ of $\mu_1, \mu_2, \dots; \sigma_1, \sigma_2, \dots; \pi_1, \pi_2, \dots, \pi_k; c_1, c_2, \dots, c_N$ given data $x_1, \dots, x_n$: $P[\theta, c|\{x\}] = \frac{P[\{x\}|\theta, c]P[c|\theta]P[\theta]}{P[\{x\}]}$ and $\log P[\{x\}|\theta, c] + \log P[c|\theta] = \sum_1^N \pi_{c_n} + \frac{(x-\mu)^2}{\sigma^2} + \log \frac{1}{\sqrt{2\pi\sigma^2}}$ and $P[\{x\}] = \sum_c \int \dots d\mu_1 \dots d\sigma_1 \dots$

Monte-Carlo methods: We want to find $P(x) = \frac{P^*(x)}{Z} = \frac{e^{-E(x)}}{Z}$ and evaluate $E(x)$ at any x . Make samples $x^{(r)}$ and estimate the expectation $\sum_x \phi(x)P(x)$.

Note: If $\{x^{(r)}\}$ is $P(x)$ then can estimate expectation $F = \langle f(x) \rangle_P, \hat{F} = \frac{\sum_{r=1}^R f(x^{(r)})}{R}$.

Q is uniform sample density. Then mean = $\frac{\sum_1^R \frac{P^*(x^{(r)})}{Q(x^{(r)})} \phi(x^{(r)})}{\sum_1^R \frac{P^*(x^{(r)})}{Q(x^{(r)})}}$.

Expected value of $\frac{P^*(x^{(r)})}{Q(x^{(r)})} \phi(x^{(r)}) = \int \frac{P^*(x)}{Q(x)} \phi(x) Q(x) dx$

Markov chain Monte-Carlo methods: Consider a parameter $Q(x; x')$ where x is next location and x' is current location. At time t at $x^{(t)}$ We do the following:

1. Generate $x \sim Q(x; x^{(t)})$
2. Compute acceptance ratio $a = \frac{P^*(x)Q(x^{(t)}; x)}{P^*(x^{(t)})Q(x; x^{(t)})}$
3. Accept x means $x^{(t+1)} = x$ and reject x means $x^{(t+1)} = x^{(t)}$. If $a > 1$ we accept x otherwise we accept x with probability a .

Example: $P^*(x) = 1$ if $x \in \{0, 1, \dots\}$ 0 otherwise. Take the graph from 0 to 20. $Q(x; x') = \frac{1}{2}$ for $x = x' - 1$ and $\frac{1}{2}$ for $x = x' + 1$. So step size parameter $\epsilon = 1$. Distance after time T $\Delta x = \sum_{t=1}^T S_t$, where $S_t = \pm 1$. $Var(\Delta x) = \langle \Delta x^2 \rangle = \sum_1^T \langle S_t^2 \rangle = T$. So minimum number of steps required for a fresh sample $T = (\frac{L}{\epsilon})^2$, where L = the length of the main coordinate axis. Here $L = 20$.

For 2 dim, we can sample from conditional distribution. At $x^{(t)}$, $x_1^{(t+1)} \sim P[x_1|x_2^{(t)}]$ and $x_2^{(t+1)} \sim P[x_2|x_1^{(t)}]$.

Hamiltonian Monte-Carlo methods: Define $P(x, p) = P(x) \cdot \frac{e^{-\frac{1}{2} \cdot \frac{p^2}{m}}}{Z_f} = \frac{e^{-E(x) - \frac{1}{2} \cdot \frac{p^2}{m}}}{Z}$, where p = momentum. Now $\dot{x} \sim p$ and $\dot{p} \sim \frac{dE}{dx}$. So here is the algorithm:

1. Simulate Newton's law. (for a time)
2. Accept/reject based on the change in total energy.
3. Randomize the momentum $f \sim e^{-\frac{1}{2} \cdot \frac{p^2}{m}}$.

4.3.2 Variational methods

Our goal is to approximate $P(x)$ by a simpler distribution $Q(x, \theta)$ and adjust θ to get the best approximation. So $\sum_x \phi(x)P(x) = \sum_x \phi(x)Q(x, \hat{\theta})$. $D(P||Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$. Let $P(x) = \{\frac{1}{3}, \frac{1}{3}, \frac{1}{3}, \epsilon\}$ and $Q(x) = \{\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4}\}$. Then $D(P||Q) = \log \frac{4}{3}$ and $D(Q||P) = \frac{3}{4} \log \frac{3}{4} + \frac{1}{4} \log \frac{1}{4\epsilon}$, as ϵ close to 0 so $D(Q||P)$ is much bigger. $D(Q||P) = \sum_x Q(x) \log \frac{Q(x)}{P(x)} = \sum_x Q(x) E(x) + \log_e Z - H_Q(x)$, where $P(x) = \frac{e^{-E(x)}}{Z}$. To minimize this, we have to minimize the fn $Q(x)$ and we can control the entropy $H_Q(x)$ and Z will be independent of the changes Z . Define $\tilde{F}(\theta) = \sum_x Q_\theta(x) E(x) - \sum_x Q_\theta(x) \log \frac{1}{Q_\theta(x)}$, this is called **variational of free energy**. Since $D(Q||P) \geq 0$, $\tilde{F}(\theta) \geq -\log Z$. So the model will be:

- **Objective function:** Minimize: $\tilde{F}(\theta) = \sum_x Q(x, \theta) E(x) - \sum_x Q(x, \theta) \log \frac{1}{Q(x, \theta)}$
- **Lower bound:** $\tilde{F}(\theta) \geq -\log Z$
- **Constraint:** $\tilde{F}(\theta) = D(Q||P) - \log Z$

This method is used to derive: (1) **Gaussian distribution inference**, (2) **Red distribution inference** etc.

Example: Approximate spin system whose energy function: $E(x, J) = -\frac{1}{2} \sum_{m,n} J_{m,n} x_m x_n - \sum_n h_n x_n$ with separable distribution $Q(x, a) = \frac{1}{Z_Q} \exp(\sum_n a_n x_n)$ and $P(x) = \frac{e^{-\beta E(x, J)}}{Z(\beta)}$. Now $Q(x, a) = \prod_{n=1}^N Q_n(x_n, a_n) = \frac{1}{Z_Q} \prod_n e^{a_n x_n}$. We optimize Q to minimize variational free energy. $X \in \{-1, +1\}$ so $\tilde{X}_n = q_{+1} + (-1)q_{-1} = q - (1 - q) = 2q - 1$. $\beta \tilde{F}(a) = \beta \sum_x Q(x, a) E(x, J) - \sum_x Q(x, a) \log \frac{1}{Q(x, a)} = \beta(-\frac{1}{2} \sum_{m,n} J_{m,n} \tilde{x}_m \tilde{x}_n - \sum_n h_n \tilde{x}_n) - \sum_n H_2^{(e)}(q_n)$. Minimizing wrt a we can find that $a_m = \beta(\sum_n J_{m,n} x_n + h_m)$ and $x_n = \tanh a_n$.

4.4 Data Modelling with Neural Networks

Suppose some pictures are stored in a memory of a computer and there is a query with some pictures, whether it is stored or not. Then the computer has to distinguish between several pictures, which is done by neural networks. Difference between a pigeon and a super-computer: (i) Pigeon has 10^{11} neurons and 10^{14} synapses; computer has 10^{10} bits of memory, CPU $\approx 10^6$ devices, (ii) Clock-rate of super-computer is 1000 MHz i.e., $10^{15} - 10^{19}$ device operations per second; pigeon has clock-rate 10 ms i.e., 10^{13} operations per second. Differences between pigeon brain and super-computer memory: High connectivity (1 neuron is connected to 1000 neurons), distributed operations, new data is stored in same old memories and operations are parallel for pigeon. Low connectivity (3-8) operations are serial and new memory can be added as a hardware for super-computer. Pigeon is robust but super-computer is not.

4.4.1 Parallel Distributed Processing

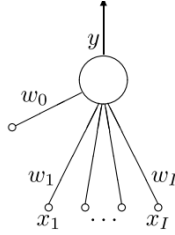


Figure 39.1. A single neuron

The above picture is the structure neurons. It is feedforward i.e., several neurons are connected to each other and all signals are moving forward. It has feedback system i.e., the output of several neurons are the input of some neurons.

Def1: Let input weights of the segments $X_1, \dots, X_k$ are $W_1, \dots, W_k$ and W_0 is the bias (virtual input) then the activation $a = \sum_i W_i X_i + W_0$.

Def2: The output $y = f(a)$, where $f(a) = \frac{1}{1+e^{-a}}$ or $f(a) = \tanh(a)$ or f is step function.

Def3: A data is of the form $\{x^{(n)}, t^{(n)}\}_{n=1}^N$ where $x^{(n)}$ is the input and $t^{(n)}$ is the target.

Our goal is to adjust weights s.t. $y(x^{(n)}, w)$ is close to $t^{(n)}$. The subjective function $G(w) = -\sum_1^N t_n \log y_n + (1-t_n) \log 1 - y_n$, where $y_n = y(x^{(n)}, w)$. G is the information content and the description length of the data. The **gradient** of G is $g = \frac{dG}{dw} = \sum_n (t_n - y_n)x^{(n)}$ and **gradient descent** is $\Delta w = -\eta g$, where η is the learning rate.

Learning with regularization (weight decay): $M(w) = G(w) + \alpha E(w)$, where $E(w) = \frac{1}{2} \sum_1^K w_n^2$ and α is the weight decay rate.

Single neuron can do digit classification with error rate 10%. A single neuron with K inputs can almost certainly memorize upto $2K$ random binary labels. So capacity of a neuron is 2 bits per connection.

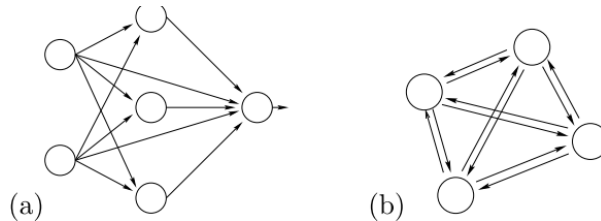
Learning as inference: We have $M(w) = G(w) + \alpha E(w)$. $P[W|\{t_n\}, \dots] = \frac{e^{-M(w)}}{Z} = \frac{P[\{t_n\}|W] \cdot P[W|\dots]}{Z}$, where $P[\{t_n\}|W] = \frac{e^{-G(w)}}{Z_1}$ and $P[W|\dots] = \frac{e^{-\alpha E(w)}}{Z_2}$. The inference which gives us prediction $P[t_{N+1}|x_{N+1}, \text{Data}, \dots] = \int P[w|\text{Data}]y(x_{N+1}, w)dw$

Applications: (i) Reading aloud (Nettalk), (ii) Handwriting recognition (LeNet), (iii) Focussing multiple-mirror telescopes, (iv) Captcha (given in the web browsers for human verification).

4.4.2 Content addressable memory

We need to make a dynamical memory system with (i) 25 dynamic variables, 300 parameters, (ii) attracting fixed points at desired memories, (iii) such that noisy versions are automatically cleaned up, (iv) with that new memory can be added incrementally and (v) robust to the corruption of more than half of the parameters.

4.4.3 Hopfield Network



Feedforward network: All the connections are directed s.t. the network forms directed acyclic graph.

Feedback network: Any network which is not feedforward network is called feedback network.

Def1: Symmetric weights network $w_{i,j} = w_{j,i}$ and $w_{i,i} = 0$.

Activity rule: Continuous Hopfield network: $a_i = \sum_j w_{i,j}x_j + \theta_i$, $x_i = \tanh(a_i)$. θ_i is the bias.

Binary Hopfield network: $x_i = -1$ if $a_i < 0$ and $x_i = +1$ if $a_i \geq 0$.

Learning Rule: If N = number of patterns to learn then $w_{i,j} = \sum_{n=1}^N x_i^{(n)}x_j^{(n)}$ (Hebb Rule)

Capacity of Hopfield network: If I = number of neurons and N = number of patterns then number of parameters $\sim \frac{I^2}{2}$ and $N_{critical} = 0.138I$. So number of bits we can learn $= N_{critical}I = 0.138I^2$. The ratio of number of bits and number of parameters = 0.24. So 0.24 bits per weight. Capacity for random patterns $= N_{critical} = 0.138I$.

Chapter 5

Conclusion

Cryptography and information theory are closely intertwined fields that play a critical role in ensuring secure communication and data protection in our increasingly digital world.

Cryptography focuses on the design of algorithms and protocols that enable secure transmission of information, protecting data from unauthorized access, alteration and destruction. It uses various techniques such as encryption, decryption, hashing and digital signatures to maintain confidentiality, integrity, authenticity and non-repudiation of information.

Information theory deals with the quantification, storage and communication of information, addressing concepts such as entropy, redundancy and channel capacity. It helps in optimizing data compression, error detection and correction, which are essential for efficient and reliable data transmission.

Information theory informs the limits of secure communication by defining the maximum amount of information that can be transmitted over a channel with a given level of security. Cryptographic protocols leverage these principles to design systems that can operate at these theoretical limits.

In conclusion, the integration of cryptography and information theory is crucial for advancing secure communication technologies. As cyber threats evolve, ongoing research in these fields is developing new methods and protocols that ensure the confidentiality, integrity and availability of information in an increasingly interconnected world. The continued interplay between these disciplines promises to enhance the security and efficiency of future communication systems.

Bibliography

- 1) Joy of Cryptography - Mike Rosulek.
- 2) A Graduate Course in Applied Cryptography - Dan Boneh and Victor Shoup.
- 3) Information-theoretic Cryptography - Himanshu Tyagi and Shun Watanabe.
- 4) Information theory, Inference and Learning Algorithms - David J.C. MacKay.